РК 1 — ответы на вопросы по лекциям

Oct 23, 2026 · @devowt

Ответы на 27 вопросов РК 1 собраны только из текста и схем лекций (темы 1–4 и 8), формулировки лекций не изменены.

Вопрос 1

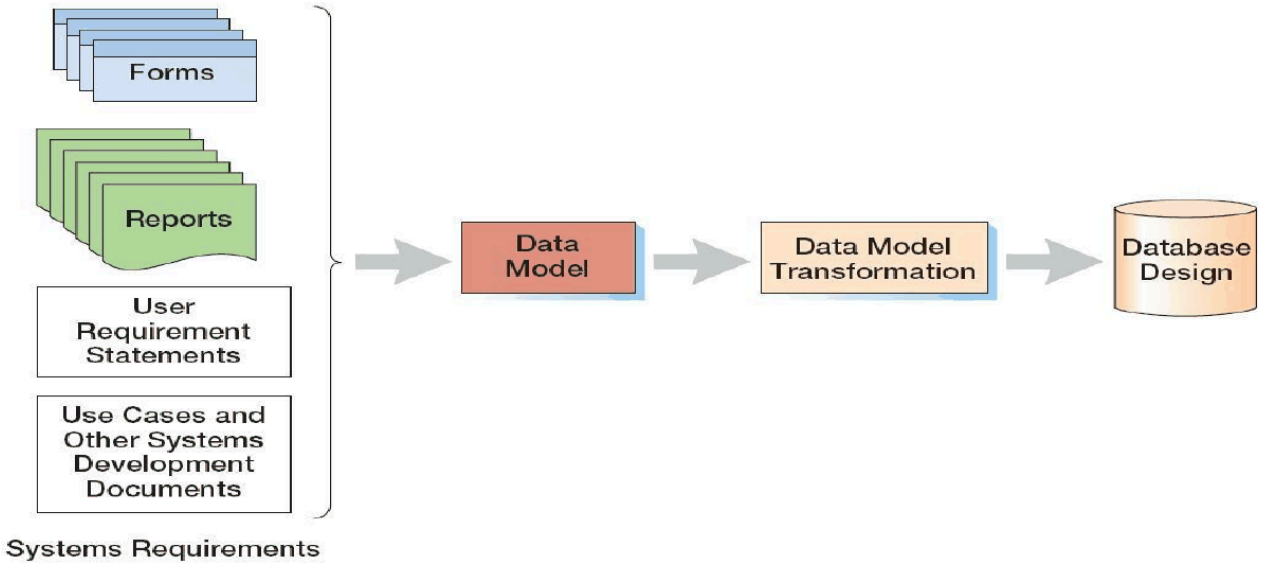
1.1. Процесс разработки баз данных и уровни моделирования информационных систем. Моделирование данных. Модель «сущность-связь» и ее основные элементы (сущность, связь, атрибуты).

Источник: Тема 2 «Моделирование данных. Модель сущность-связь».

Процесс разработки БД

- общие стратегии
  - разработка сверху вниз (top-down database development)
    - получение абстрактной модели данных исходя из анализа стратегических целей организации, способов их достижения, а также информации и способов ее представления, которые необходимы для достижения этих целей;
    - лучшее взаимодействие подсистем, глобальный охват;
  - разработка снизу вверх (bottom-up database development)
    - для разработки выбирается конкретная система, которая выполняет только часть функций предприятия;
    - исходя из сформированных требований, выбранная подсистема создается быстрее и с меньшими рисками;
- одна из основ эффективной реализации приложения базы данных – ясное представление модели предметной области

Три уровня моделей информационных систем



- внешние модели (external schema): представление пользователей о системе (user view);
- концептуальная модель (conceptual schema): абстрактное представление системы (данных);
- внутренние модели (internal schema).

Модели данных (ANSI)

- внешняя модель (external schema) – набор представлений пользователей о той части предметной области, с которой он сталкивается;
- концептуальная модель (conceptual schema / conceptual data model) – набор ключевых объектов предметной области, их взаимосвязей (взаимодействий) и ограничений, накладываемых на объекты;
- логическая модель (logical schema / logical data model / information schema) – представление концептуальной модели в соответствии с логической моделью, используемой для хранения данных;
- физическая модель (physical data model) – описание физического представления, хранения и обработки данных.

Моделирование данных

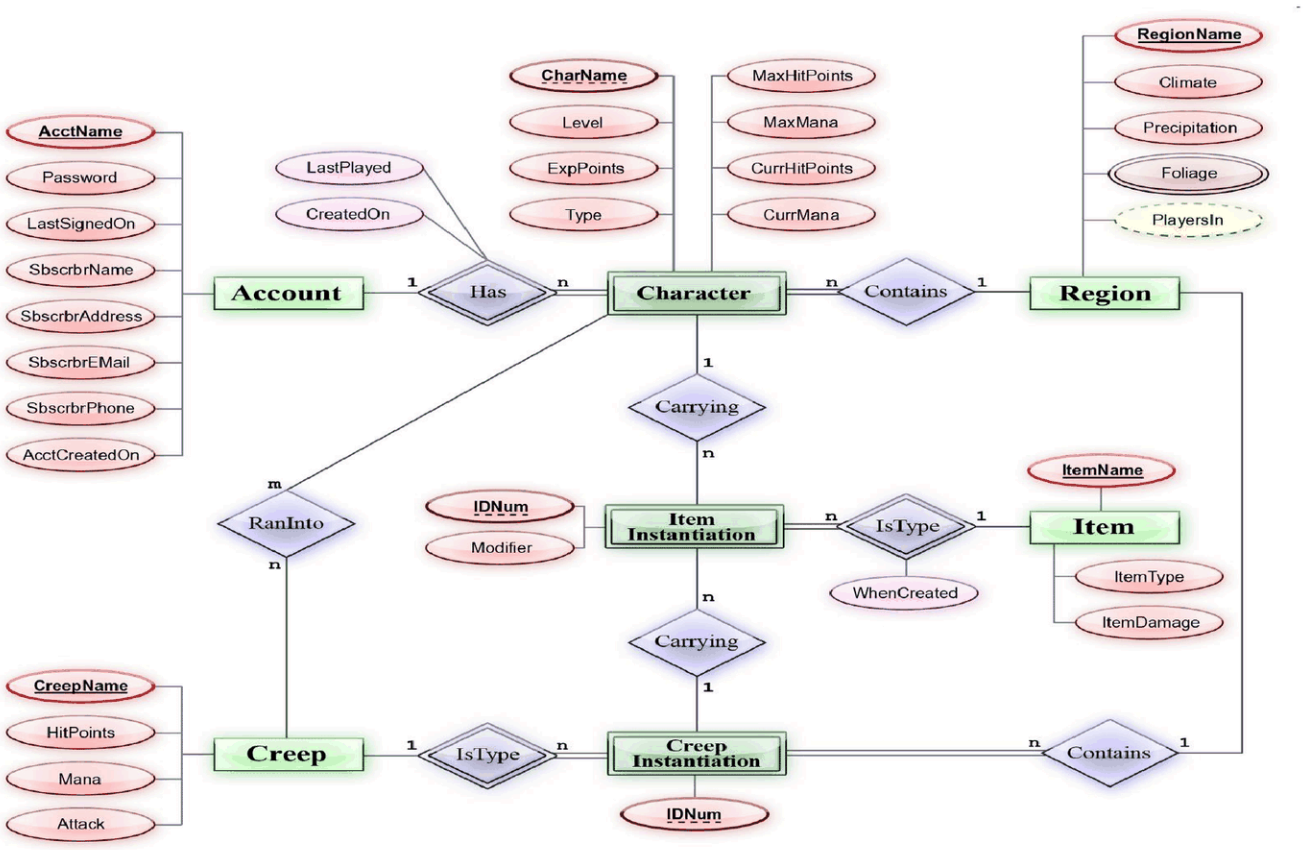
- база данных является «моделью модели», то есть через объекты базы данных описывает представление пользователей о конкретной предметной области;

- при любом процессе разработки необходимо сформулировать требования и построить на их основе модель данных, что является во многом творческой задачей, решение которой основано на опыте и интуиции;
- моделирование данных (data modeling) - процесс создания логического представления базы данных (пользовательская модель данных /user data model/, модель требований к данным /requirements data model/);
- модель данных представляет собой совокупность понятий и графических символов для построения концептуальных схем, т.е. содержит языковые и изобразительные стандарты для своего представления;
  - модель сущность – связь (entity-relationship model, E-R model);
  - семантическая объектная модель (semantic object model);

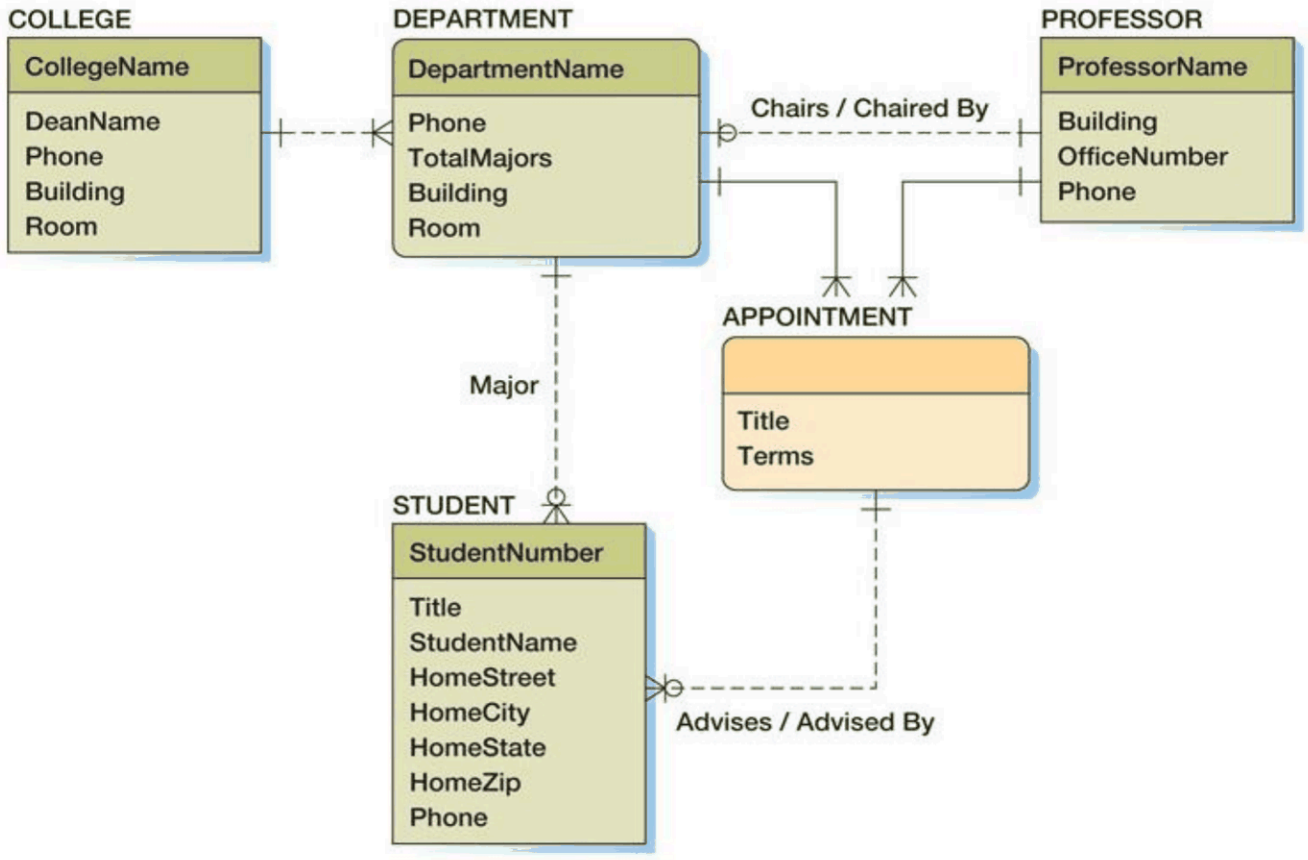
ER-модель: варианты

- исходная ER-модель: предложена Питером Ченом в 1976 году ("The Entity-Relationship Model--Toward a Unified View of Data". In: ACM Transactions on Database Systems 1/1/1976 ACM-Press ISSN 0362-5915, S. 9–36);
- расширенная ER-модель: дополненная модель П.Чена;
- информационная инженерия (Information Engineering, IE) – предложена Джеймсом Мартином в 1990 году, использует нотацию “crow’s foot”;
- IDEF1X (Integrated DEFinition, 1, eXtended) – национальный стандарт (NIST, National Institute of Standards and Technology - http://www.itl.nist.gov/fipspubs/idef1x.doc);
- Unified Modeling Language (UML, универсальный язык моделирования) – стандарт OMG (Object Management Group).

ER-модель: пример (нотация П.Чена)

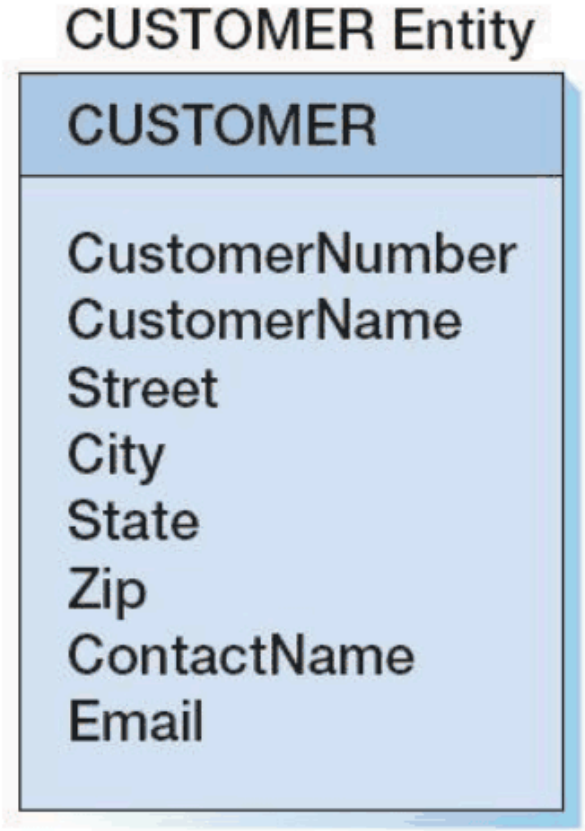


Модель сущность-связь (нотация Дж. Мартина)



Сущности

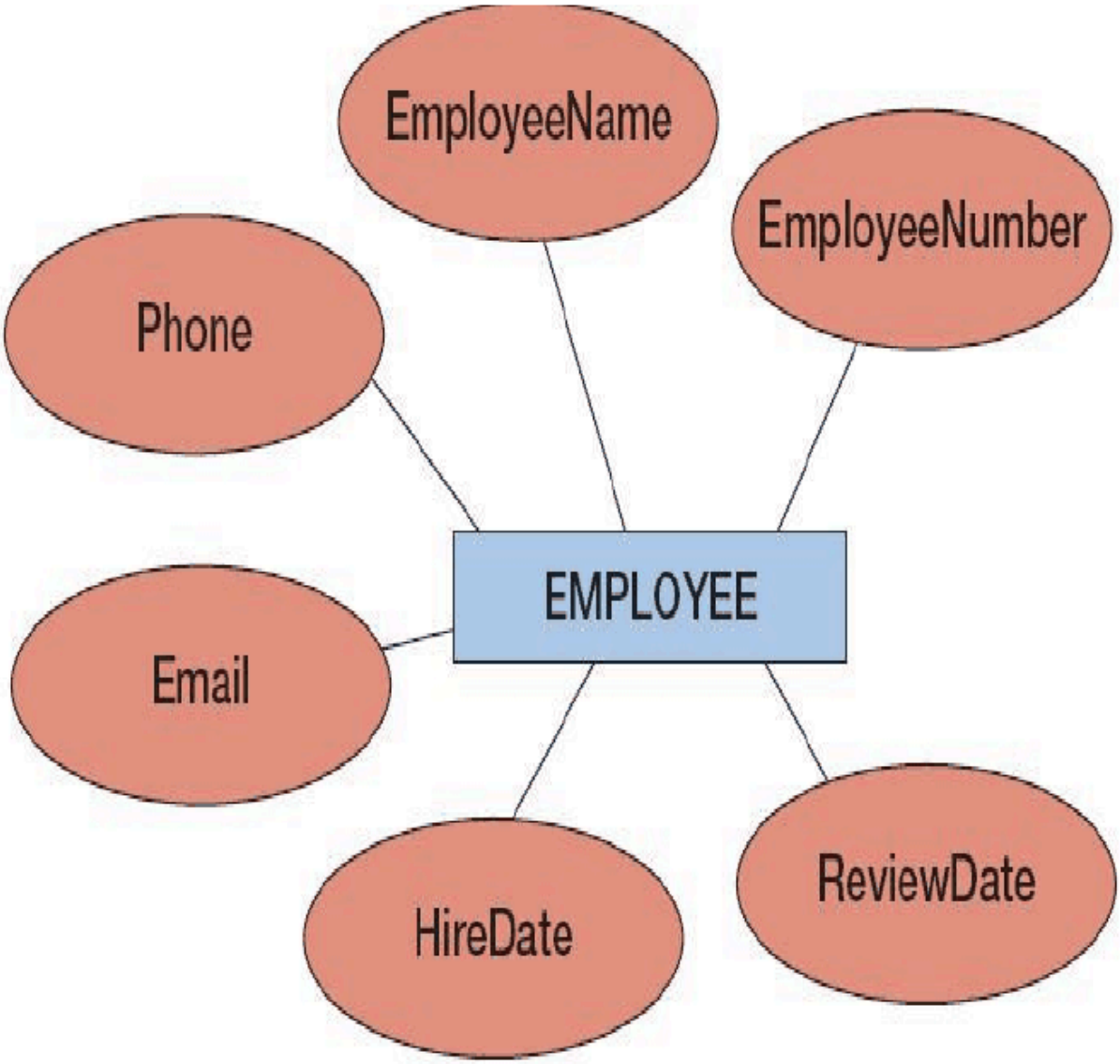
- сущность (entity) – некоторый объект, который способен существовать независимо и может быть идентифицирован;
  - класс сущностей – набор сущностей определенного типа;
  - экземпляр сущности – сущность некоторого класса с заданными значениями атрибутов;
- сущность может являться абстракцией:
  - реального физического объекта;
  - события, действия или процесса (заказ, услуга, транзакция).

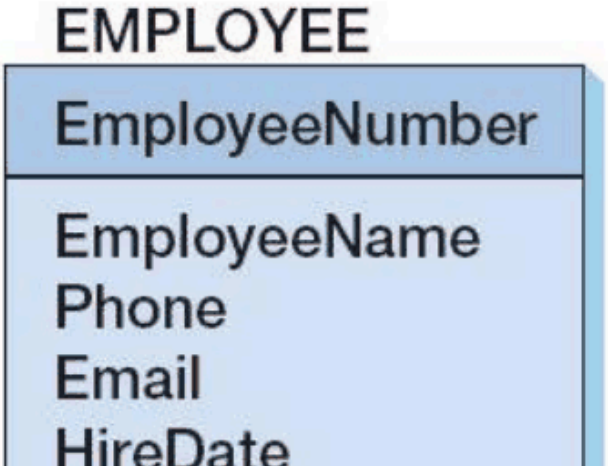

CUSTOMER Entity
CUSTOMER
CustomerNumber
CustomerName
Street
City
State
Zip
ContactName
Email

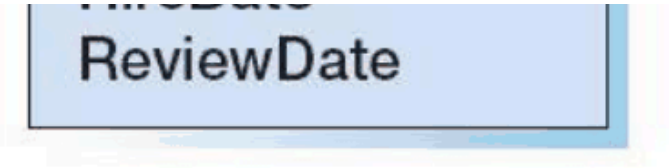

Two CUSTOMER Instances
1234
Ajax Manufacturing
123 Elm Street
Memphis
TN
32455
P_Schwartz
P_S@Ajax.com
99890
Jones Brothers
434 10th Street
Boston
MA
01234
Fritz Billingsley
Fritz@JB.com

Атрибуты

- атрибут (attribute) – свойство, которым обладает сущность;
- все сущности определенного класса обладают одинаковым набором атрибутов, но различаются значениями данных атрибутов;
- в общем случае, атрибуты могут быть:
  - составными (composite);
  - многозначными(multi-value).
- идентификатор сущности (entity identifier) – набор атрибутов, которые определяют, именуют сущность;
- идентификаторы могут быть составными, т.е. состоять более, чем из одного атрибута;
- заметим, идентификаторы в модели данных становятся ключами в физических моделях;


EmployeeName
EmployeeNumber
Phone
EMPLOYEE
Email
HireDate
ReviewDate


EMPLOYEE
EmployeeNumber
EmployeeName
Phone
Email
HireDate



Связи

- связь (relationship) определяет, как сущности соотносятся друг с другом;
- подобно классам и экземплярам сущностей, можно выделить классы связей и экземпляры связей;
- степень (degree) связи показывает, сколько сущностей определяют эту связь (участвуют в связи):
  - бинарные связи (binary);
  - тернарные связи (ternary);
- оригинальная ER-модель позволяет назначать атрибуты также и связям, но в более современных нотациях от такой возможности отказались;
- ER-модель для описания данных не оперирует понятиями ключей, что упрощает создание модели в условиях неопределенности на начальных этапах проектирования.



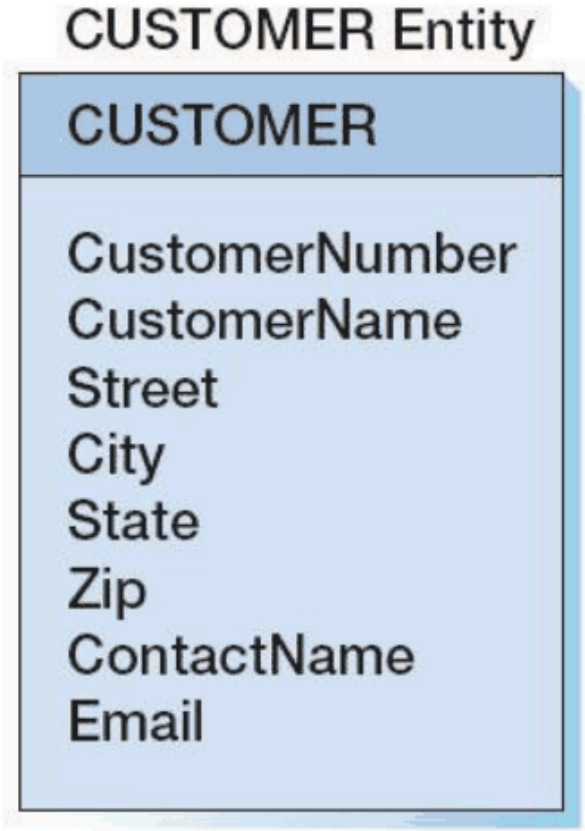
1.2. Модель «сущность-связь» и ее основные элементы (сущность, связь, атрибуты). Кардинальные числа связей.

Источник: Тема 2 «Моделирование данных. Модель сущность-связь».

Сущности

- сущность (entity) – некоторый объект, который способен существовать независимо и может быть идентифицирован;
  - класс сущностей – набор сущностей определенного типа;
  - экземпляр сущности – сущность некоторого класса с заданными значениями атрибутов;

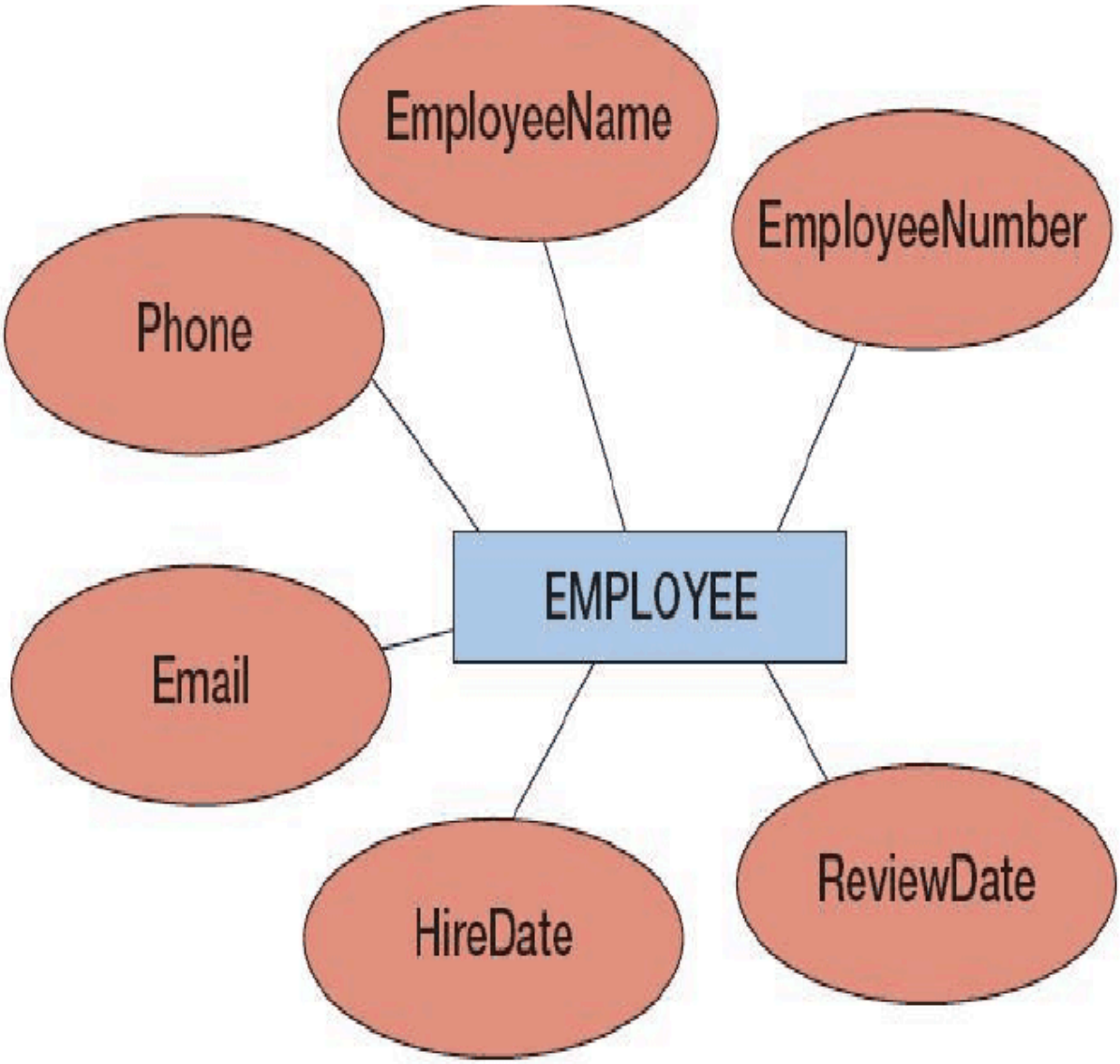
- сущность может являться абстракцией:
  - реального физического объекта;
  - события, действия или процесса (заказ, услуга, транзакция).

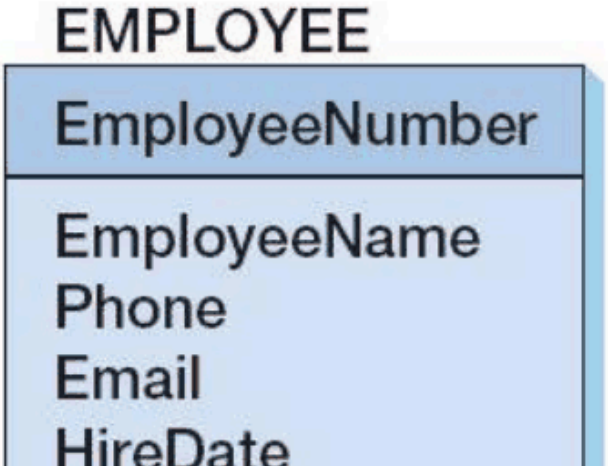

CUSTOMER Entity
CUSTOMER
CustomerNumber
CustomerName
Street
City
State
Zip
ContactName
Email

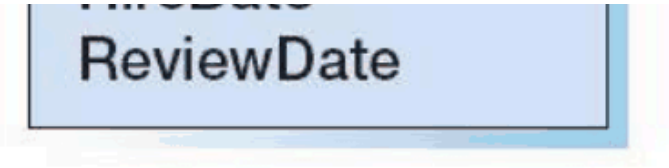

Two CUSTOMER Instances
1234
Ajax Manufacturing
123 Elm Street
Memphis
TN
32455
P_Schwartz
P_S@Ajax.com
99890
Jones Brothers
434 10th Street
Boston
MA
01234
Fritz Billingsley
Fritz@JB.com

Атрибуты

- атрибут (attribute) – свойство, которым обладает сущность;
- все сущности определенного класса обладают одинаковым набором атрибутов, но различаются значениями данных атрибутов;
- в общем случае, атрибуты могут быть:
  - составными (composite);
  - многозначными(multi-value).
- идентификатор сущности (entity identifier) – набор атрибутов, которые определяют, именуют сущность;
- идентификаторы могут быть составными, т.е. состоять более, чем из одного атрибута;
- заметим, идентификаторы в модели данных становятся ключами в физических моделях;


EmployeeName
EmployeeNumber
Phone
EMPLOYEE
Email
HireDate
ReviewDate


EMPLOYEE
EmployeeNumber
EmployeeName
Phone
Email
HireDate



Связи

- связь (relationship) определяет, как сущности соотносятся друг с другом;
- подобно классам и экземплярам сущностей, можно выделить классы связей и экземпляры связей;
- степень (degree) связи показывает, сколько сущностей определяют эту связь (участвуют в связи):
  - бинарные связи (binary);
  - тернарные связи (ternary);
- оригинальная ER-модель позволяет назначать атрибуты также и связям, но в более современных нотациях от такой возможности отказались;
- ER-модель для описания данных не оперирует понятиями ключей, что упрощает создание модели в условиях неопределенности на начальных этапах проектирования.



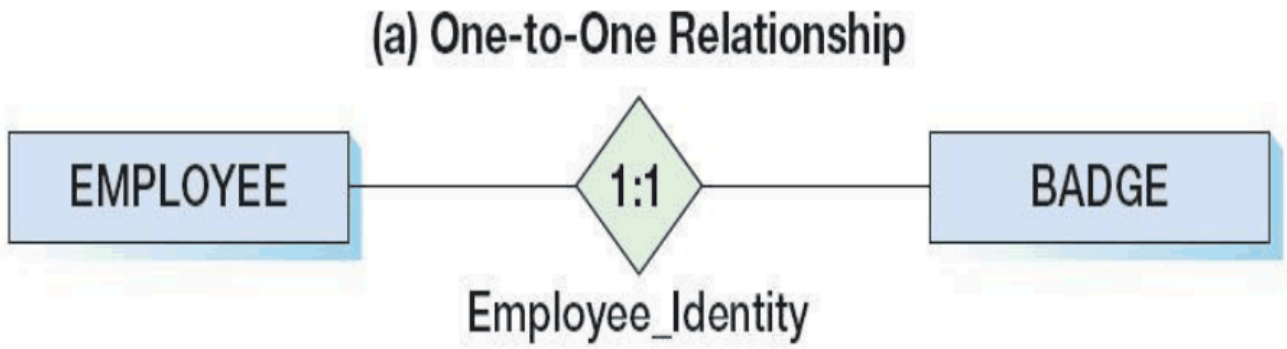
Кардинальные числа связей

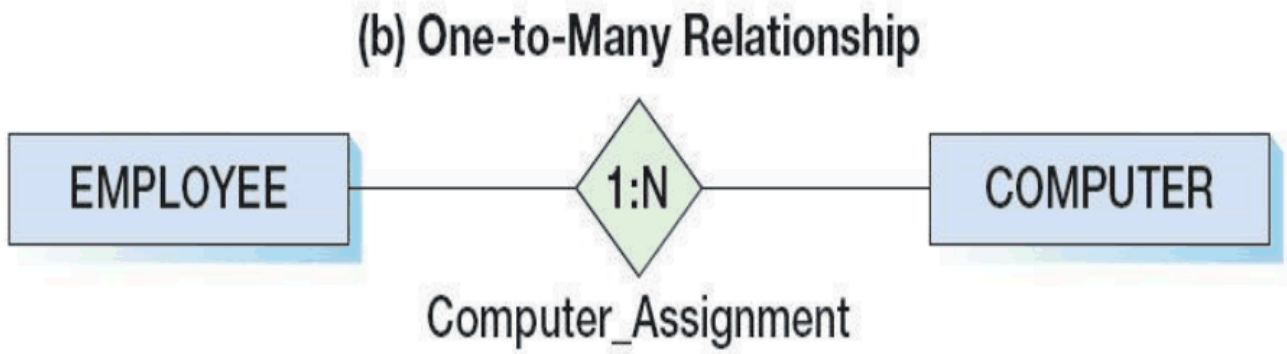
- кардинальное число (cardinality, мощность) указывает число экземпляров сущностей, которые участвуют в связи;
- максимальное кардинальное число (maximum cardinality) - максимальное количество экземпляров сущностей, которые могут участвовать в связи;
- минимальное кардинальное число (minimum cardinality) - минимальное количество экземпляров сущностей, которые должны участвовать в связи.

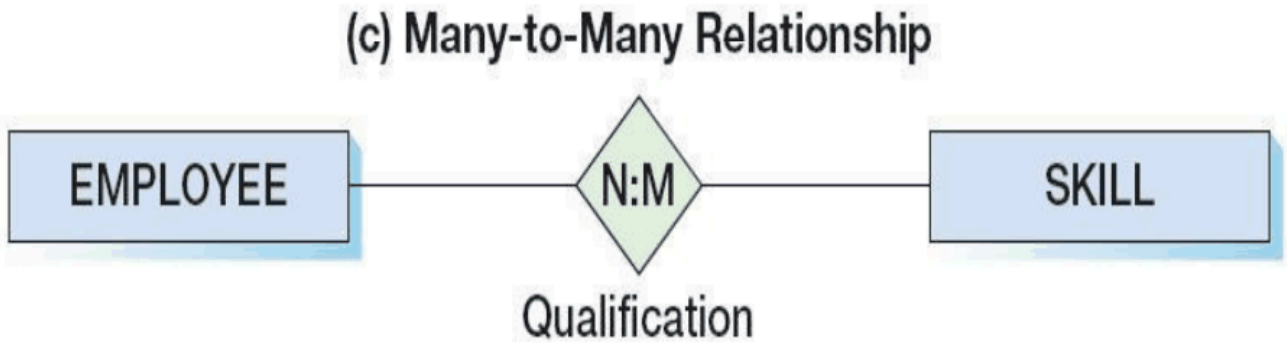
Максимальное кардинальное число

- существует три типа бинарных связей, в зависимости от максимального кардинального числа:

  - связь типа «один-к-одному» (one-to-one): [1:1];
  - связь типа «один-ко-многим» (one-to-many): [1:N];
  - связь типа «многие-ко-многим» (many-to-many): [N:M];
- рассматриваемые связи также называются связями типа принадлежность (HAS-A relationships);
- в связях типа «один-ко-многим» сущность на единичной стороне часто называют родительской (parent), а на множественной стороне – дочерней (child).

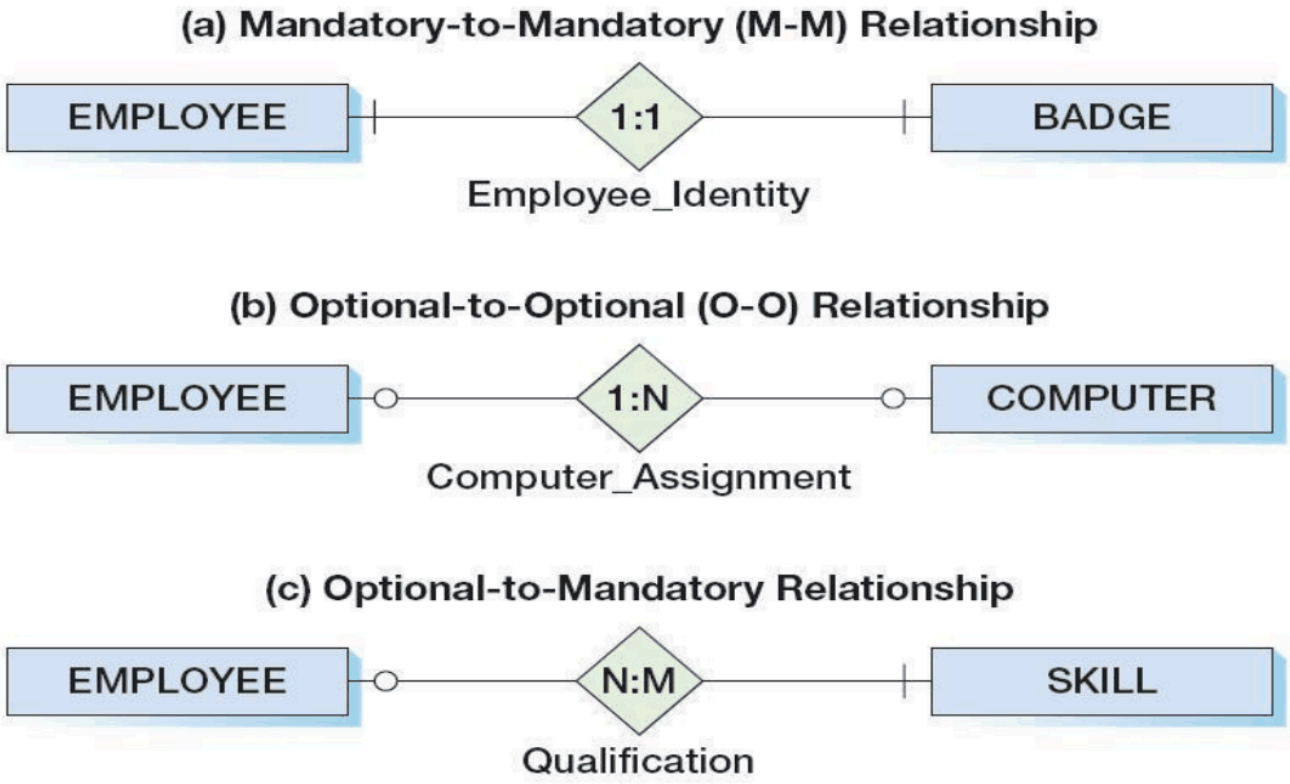




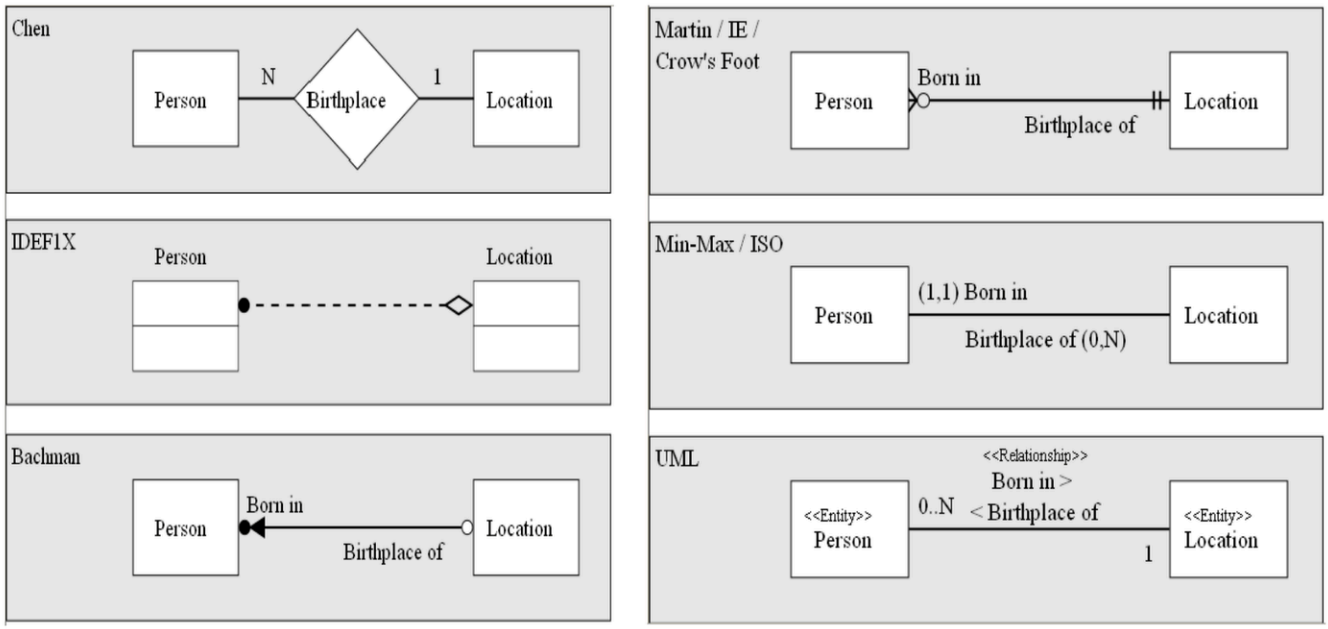


Минимальное кардинальное число

- минимальное кардинальное число, как правило, указывается как ноль или единица:
  - если минимальное каридинальное число равно 0, то участие некоторого экземпляра сущности в связи не является обязательным (optional);
  - если минимальное каридинальное число равно 1, то участие некоторого экземпляра сущности в связи является обязательным (mandatory);

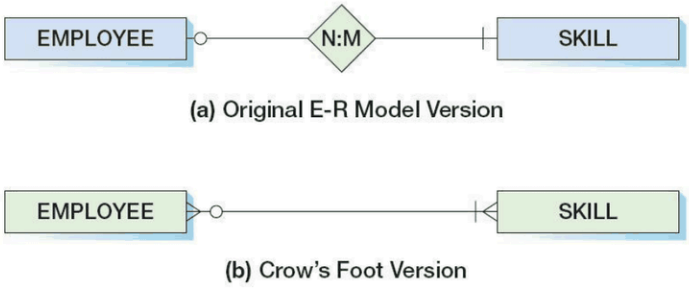


Представление бинарных связей в различных нотациях

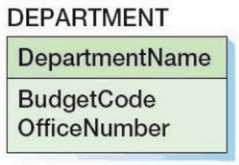
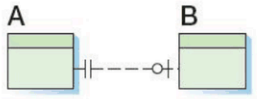
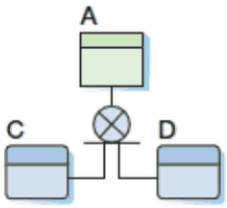
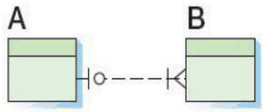
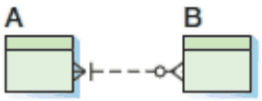
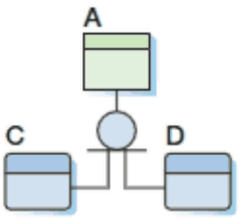
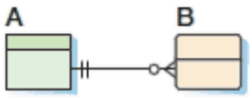


Нотация Information Engineering (crow's foot, **Дж.Мартин**)

Symbol	Meaning
	One—Mandatory
	Many—Mandatory
	One—Optional
	Many—Optional



Резюме по нотации

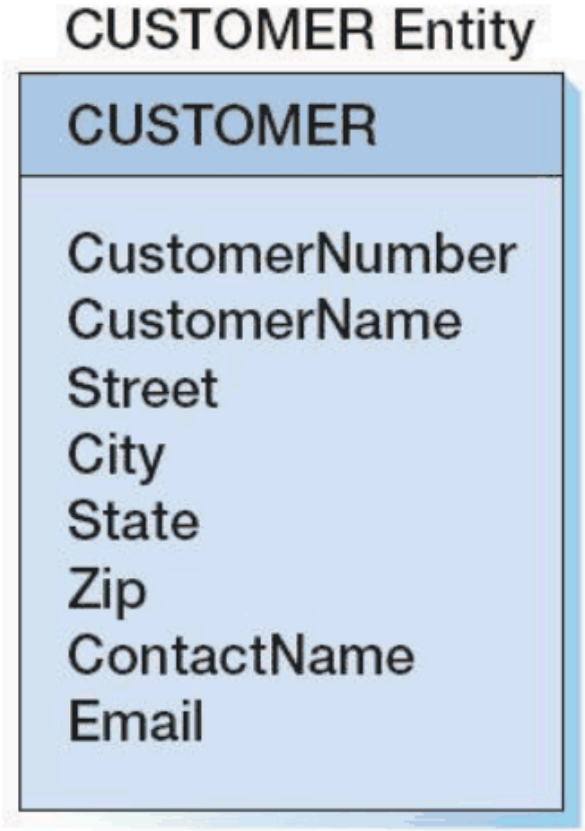
<table>
<tr><td colspan="2"> 
</td><td colspan="2">сущность DEPARTMENT;
DepartmentName – идентификатор;
BudgetCode,OfficeNumber – атрибуты;
Идентификационно-зависимые сущности изображаются прямоугольниками со скругленными углами</td></tr>
<tr><td> 
</td><td>неидентифицирующая связь 1:1
сущности A соответствует 0 или 1 сущности B,
сущности B соответствует ровно одна сущность A</td><td rowspan="2"> 
</td><td rowspan="2">сущность A является супертипом,
C и D являются взаимоисключающими подтипами,
дискриминатор не указан</td></tr>
<tr><td> 
</td><td>неидентифицирующая связь 1:N
сущности A соответствует 1..N сущностей B,
сущности B соответствует 0 или одна сущность A</td></tr>
<tr><td> 
</td><td>неидентифицирующая связь M:N
сущности A соответствует 0..N сущностей B,
сущности B соответствует 0..M сущностей A</td><td rowspan="2"> 
</td><td rowspan="2">сущность A является супертипом,
C и D являются невзаимоисключающими подтипами</td></tr>
<tr><td> 
</td><td>идентифицирующая связь 1:N
сущности A соответствует 0..N сущностей B,
сущности B соответствует ровно одна сущность A</td></tr>
</table>

1.3. Модель «сущность-связь». Идентификационно-зависимые, слабые и сильные сущности. Сущности категория-подтип.

Источник: Тема 2 «Моделирование данных. Модель сущность-связь».

Сущности

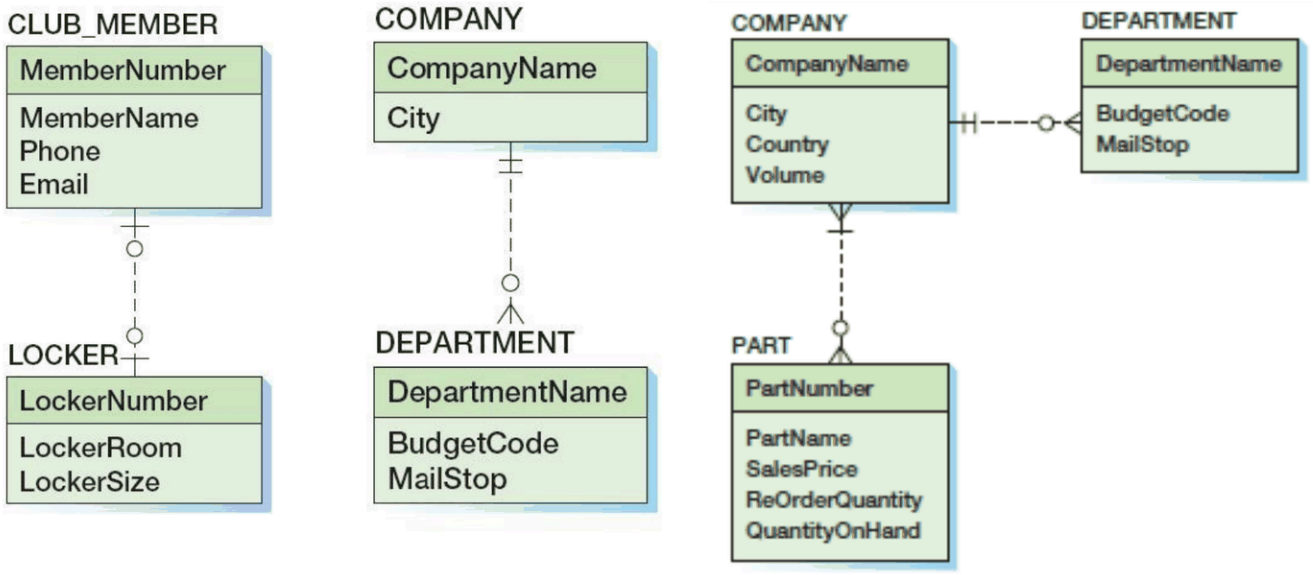
- сущность (entity) – некоторый объект, который способен существовать независимо и может быть идентифицирован;
  - класс сущностей – набор сущностей определенного типа;
  - экземпляр сущности – сущность некоторого класса с заданными значениями атрибутов;
- сущность может являться абстракцией:
  - реального физического объекта;
  - события, действия или процесса (заказ, услуга, транзакция).





Сильные сущности: примеры

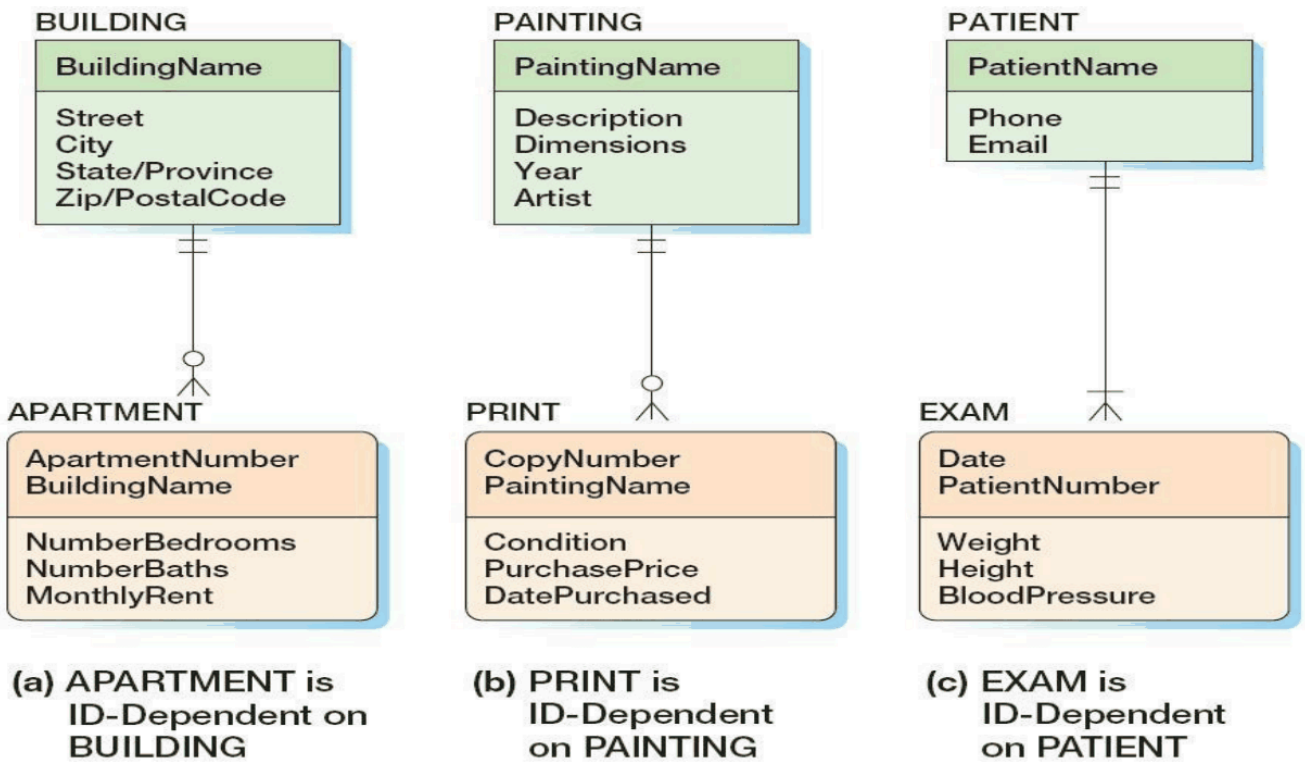
- связь 1:1;
- связь 1:N;
- связь N:M.



- связь 1:1;

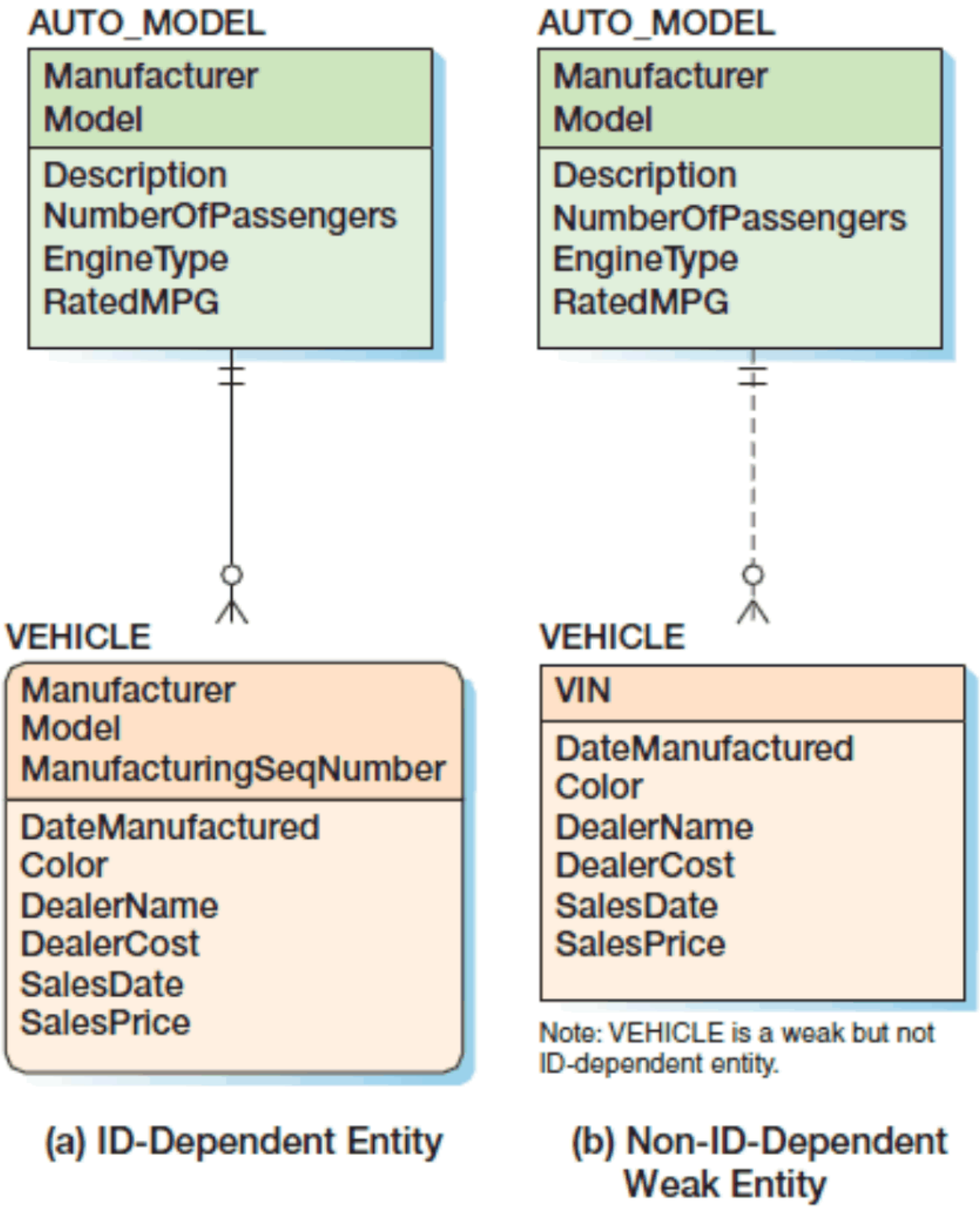
Идентификационно-зависимые сущности

- идентификационно-зависимой (ID-dependent) называется такая дочерняя сущность, идентификатор которой содержит идентификатор родительской сущности (дочерняя сущность является логическим расширением или составной частью родительской);
- минимальное кардинальное число родительской сущности для идентификационно-зависимой сущности всегда равно единице.

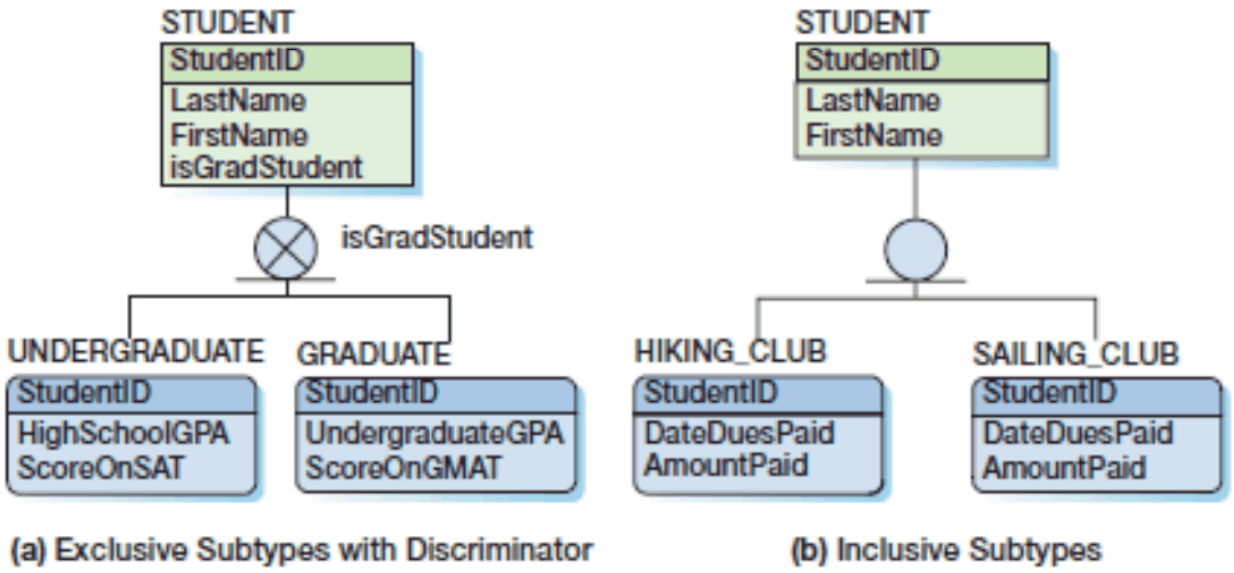


Слабые сущности

- слабой сущностью (weak entity) называется сущность, существование которой зависит от наличия родительской сущности;
- все идентификационно-зависимые сущности являются слабыми;
- возможно наличие в модели слабых сущностей, которые при этом не являются идентификационно-зависимыми, в этом случае условия зависимости реализуются дополнительными средствами бизнес-логики.



Сущности категория-подтип



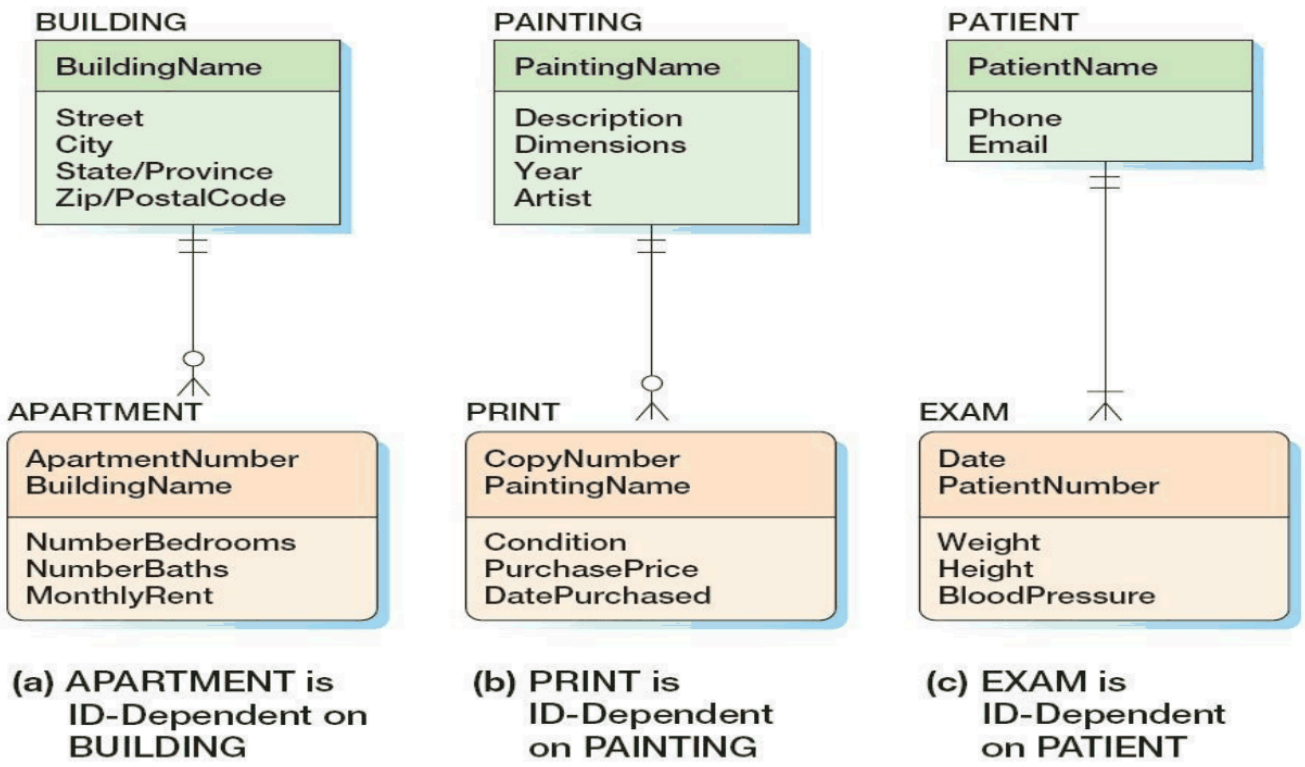
- родительская сущность (supertype entity) содержит атрибуты, общие для нескольких подтипов;
- подтип (subtype entity) расширяет набор атрибутов родительской сущности;
- в ряде случаев один из атрибутов супертипа (дискриминатор, discriminator) указывает на то, каким из подтипов является конкретный экземпляр сущности;
- подтипы могут быть взаимоисключающими (exclusive) и невзаимоисключающими (inclusive).

1.4. Модель «сущность-связь»: шаблоны моделирования типа «сопряжение» и «прототип-экземпляр», представление многозначных атрибутов и рекурсивных связей.

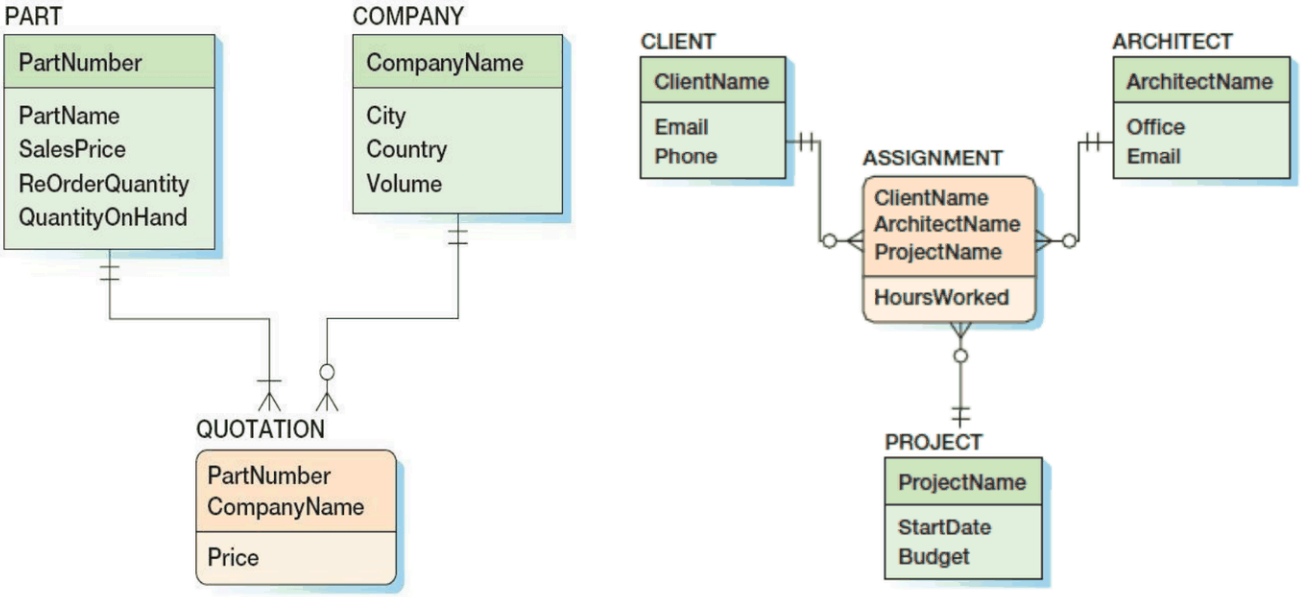
Источник: Тема 2 «Моделирование данных. Модель сущность-связь».

Идентификационно-зависимые сущности

- идентификационно-зависимой (ID-dependent) называется такая дочерняя сущность, идентификатор которой содержит идентификатор родительской сущности (дочерняя сущность является логическим расширением или составной частью родительской);
- минимальное кардинальное число родительской сущности для идентификационно-зависимой сущности всегда равно единице.



Идентификационно-зависимые сущности: шаблон «сопряжение» (The Association Pattern)



Идентификационно-зависимые сущности: шаблон «прототип-экземпляр» (The Archetype/Instance Pattern)

- данный шаблон необходим в том случае, когда идентификационно-зависимая дочерняя сущность является физической реализацией некоторой абстрактной или логической родительской сущности.

Image — Шаблон «прототип-экземпляр» (The Archetype/Instance Pattern)

Реализация шаблона «прототип-экземпляр» с использованием слабых сущностей

Image — Реализация шаблона «прототип-экземпляр» с использованием слабых сущностей

Идентификационно-зависимые сущности: многозначные атрибуты (The Multivalued Attribute Pattern)

Image — Многозначные атрибуты (The Multivalued Attribute Pattern)

Шаблон «строка-элемент» (The Line-Item Pattern)

Image — Шаблон «строка-элемент» (The Line-Item Pattern)

Смешанные шаблоны

Image — Смешанные шаблоны

Фактически: многозначные атрибуты, связанные с сильной сущностью

Рекурсивные связи

- рекурсивная (recursive) связь возникает, когда некоторый класс сущностей имеет связь с самим собой

Image — Рекурсивные связи

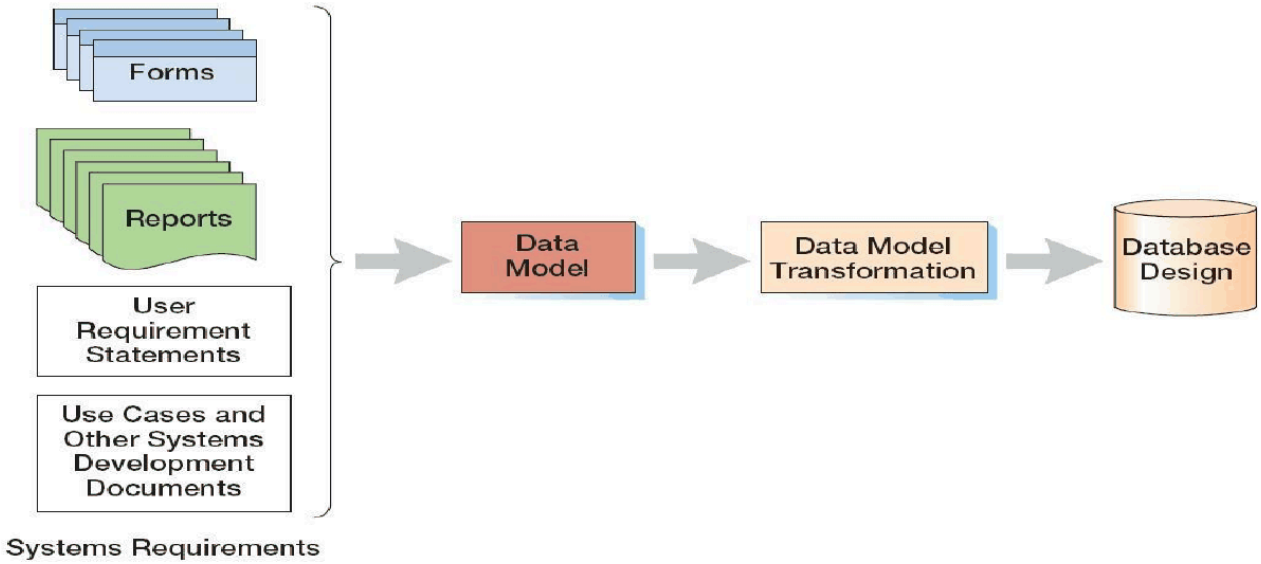
1.5. Процесс разработки баз данных и уровни моделирования информационных систем. Моделирование данных. Модель семантических объектов и ее основные элементы (объекты, атрибуты).

Источник: Тема 2 «Моделирование данных. Семантическая объектная модель».

Процесс разработки БД

- общие стратегии:
  - разработка сверху вниз (top-down database development):
    - получение абстрактной модели данных исходя из анализа стратегических целей организации, способов их достижения, а также информации и способов ее представления, которые необходимы для достижения этих целей;
    - лучшее взаимодействие подсистем, глобальный охват;
  - разработка снизу вверх (bottom-up database development):
    - для разработки выбирается конкретная система, которая выполняет только часть функций предприятия;
    - исходя из сформированных требований, выбранная подсистема создается быстрее и с меньшими рисками;
- одна из основ эффективной реализации приложения базы данных – ясное представление модели предметной области.

Три уровня моделей информационных систем



- внешние модели (external schema): представление пользователей о системе (user view);
- концептуальная модель (conceptual schema): абстрактное представление системы (данных);
- внутренние модели (internal schema).

Модели данных (ANSI)

- внешняя модель (external schema) – набор представлений пользователей о той части предметной области, с которой он сталкивается;
- концептуальная модель (conceptual schema / conceptual data model) – набор ключевых объектов предметной области, их взаимосвязей (взаимодействий) и ограничений, накладываемых на объекты;
- логическая модель (logical schema / logical data model / information schema) – представление концептуальной модели в соответствии с логической моделью, используемой для хранения данных;
- физическая модель (physical data model) – описание физического представления, хранения и обработки данных.

Моделирование данных

- база данных является «моделью модели», то есть через объекты базы данных описывает представление пользователей о конкретной предметной области;
- при любом процессе разработки необходимо сформулировать требования и построить на их основе модель данных, что является во многом творческой задачей, решение которой основано на опыте и интуиции;
- моделирование данных (data modeling) - процесс создания логического представления базы данных (пользовательская модель данных /user data model/, модель требований к данным /requirements data model/);
- модель данных представляет собой совокупность понятий и графических символов для построения концептуальных схем, т.е. содержит языковые и изобразительные стандарты для своего представления;
  - модель сущность – связь (entity-relationship model, E-R model);
  - семантическая объектная модель (semantic object model);

Семантическая объектная модель

- семантическая объектная модель – альтернативный способ представления модели данных;
- семантика (от греч . semantikos - обозначающий) - значение единиц языка.

Image — Семантическая объектная модель

Семантический объект

- семантический объект (semantic object) – некоторый объект, который способен существовать независимо и может быть идентифицирован (именованный набор атрибутов, исчерпывающе описывающий отдельную сущность);
- аналогично сущностям ER-модели,
  - можно определить:
    - класс объектов (object class) – набор объектов определенного типа;
    - экземпляр объекта (object instance) – объект некоторого класса с заданными значениями атрибутов;
    - атрибут (attribute) – свойство, которым обладает объект;
  - объект может являться абстракцией:
    - реального физического объекта;
    - события, действия или процесса (заказ, услуга, транзакция).

Атрибуты

- атрибут (attribute) – свойство, которым обладает объект;
- все объекты определенного класса обладают одинаковым набором атрибутов, но различаются значениями данных атрибутов (исчерпывающее описание, sufficient description);
- в общем случае, атрибуты могут быть:
  - простыми (simple): атрибут со значением элементарного типа;
  - групповыми, или составными (group): атрибут состоит из набора простых атрибутов;
  - объектными (object): атрибут является семантическим объектом (очевидно, атрибуты данного типа являются <зеркальными> для двух соответствующих объектов).

Image — Атрибуты

1.6. Модель семантических объектов и ее основные элементы (объекты, атрибуты). Кардинальные числа атрибутов. Идентификаторы.

Источник: Тема 2 «Моделирование данных. Семантическая объектная модель».

Семантическая объектная модель

- семантическая объектная модель – альтернативный способ представления модели данных;
- семантика (от греч . semantikos - обозначающий) - значение единиц языка.

Image — Семантическая объектная модель

Семантический объект

- семантический объект (semantic object) – некоторый объект, который способен существовать независимо и может быть идентифицирован (именованный набор атрибутов, исчерпывающе описывающий отдельную сущность);
- аналогично сущностям ER-модели,
  - можно определить:
    - класс объектов (object class) – набор объектов определенного типа;
    - экземпляр объекта (object instance) – объект некоторого класса с заданными значениями атрибутов;
    - атрибут (attribute) – свойство, которым обладает объект;
  - объект может являться абстракцией:
    - реального физического объекта;
    - события, действия или процесса (заказ, услуга, транзакция).

Атрибуты

- атрибут (attribute) – свойство, которым обладает объект;
- все объекты определенного класса обладают одинаковым набором атрибутов, но различаются значениями данных атрибутов (исчерпывающее описание, sufficient description);

- в общем случае, атрибуты могут быть:
  - простыми (simple): атрибут со значением элементарного типа;
  - групповыми, или составными (group): атрибут состоит из набора простых атрибутов;
  - объектными (object): атрибут является семантическим объектом (очевидно, атрибуты данного типа являются <зеркальными> для двух соответствующих объектов).

Image — Атрибуты

Кардинальные числа атрибутов

- кардинальное число (cardinality, мощность) указывает количество значений, которые может принимать атрибут в корректном (valid) экземпляре объекта;
- максимальное кардинальное число (maximum cardinality) - максимальное количество значений, которые может принимать атрибут – как правило, 1 или N;
- минимальное кардинальное число (minimum cardinality) - минимальное количество значений, которые должен принимать атрибут – как правило, 0 или 1.
- дополнительные термины для описания атрибутов:
  - однозначный атрибут (single-value attribute) – кардинальность ..1;
  - многозначный атрибут (multivalue attribute) – кардинальность ..N;
  - необъектный атрибут (nonobject attribute) – простой или групповой.

Image — Кардинальные числа атрибутов

Домены атрибутов

- домен (domain) – множество допустимых значений атрибута;
- понятие домена имеет различный смысл для атрибутов различных типов:
  - для простых атрибутов:
    - физическое описание: тип данных и дополнительные наложенные ограничения;

        - семантическое (логическое) описание: описывает назначение данного атрибута в модели;
    - для составных атрибутов:
        - физическое описание: список и порядок входящих в него простых атрибутов;
        - семантическое описание: описывает назначение данного атрибута в модели;
    - для объектных атрибутов: множество всех экземпляров объектов рассматриваемого класса.

Идентификаторы

- идентификатор объекта (object identifier) – набор атрибутов, которые определяют, именуют сущность;
- идентификаторы могут быть составными, т.е. состоять более, чем из одного атрибута;
- обозначаются «ID» или «ID» в случае уникальности;

Image — Идентификаторы

Экземпляры семантических объектов

Image — Экземпляры семантических объектов

1.7. Типы семантических объектов. Простой объект. Составной объект. Составной объект с независимыми и вложенными группами.

Источник: Тема 2 «Моделирование данных. Семантическая объектная модель».

Типы объектов: простой объект

- простой объект (simple object) – объект, содержащий только однозначные простые или групповые атрибуты.

Image — Типы объектов: простой объект

Типы объектов: составной объект

- составной объект (composite object) – объект, содержащий один или более многозначный простой или групповой атрибуты.

Image — Типы объектов: составной объект

Составной объект

- с независимыми группами;
- с вложенными группами.

Image — Составной объект с независимыми и вложенными группами

1.8. Типы семантических объектов. Сложный объект. Гибридный объект. Ассоциативный объект.

Источник: Тема 2 «Моделирование данных. Семантическая объектная модель».

Типы объектов: сложный объект

- сложный объект (compound object) – объект, содержащий один или более объектный атрибут.

Image — Типы объектов: сложный объект

Сложный объект (1:1; 1:N; M:N)

Image — Сложный объект (1:1; 1:N; M:N)

Типы объектов: гибридный объект

- гибридный объект (hybrid object) – объект, содержащий один или более групповой атрибут, содержащий в себе объектный атрибут

Image — Типы объектов: гибридный объект

Гибридный объект: пример

Image — Гибридный объект: пример

- модификация для указания того, что позиция должна появляться в заказе не более одного раза

Типы объектов: ассоциативный объект

- ассоциативный объект (association object) – объект, сопоставляющий два или более объектов, и содержащий дополнительную информацию о таком сопоставлении

Image — Типы объектов: ассоциативный объект

1.9. Типы семантических объектов. Схема «родитель-подтип». Взаимоисключающие и вложенные подтипы. Схема «прототип-экземпляр».

Источник: Тема 2 «Моделирование данных. Семантическая объектная модель».

Типы объектов: родитель / подтип

- родительский объект (parent / supertype object) содержит атрибуты, общие для нескольких объектов-подтипов (кардинальное число определяет обязательность объекта-подтипа);
- объект-подтип (subtype object) расширяет набор атрибутов родительского объекта.

Image — Типы объектов: родитель / подтип

Родитель / подтип : взаимоисключающие подтипы

Image — Родитель / подтип : взаимоисключающие подтипы

- X.Y.Z: для взаимоисключающих типов
  - X (0/1) – минимальное кардинальное число;
  - Y – минимальное количество атрибутов со значениями;
  - Z – максимальное количество атрибутов со значениями.

Родитель / подтип : вложенные подтипы

Image — Родитель / подтип : вложенные подтипы

Типы объектов: прототип / экземпляр

- объект-прототип (archetype object) является некоторым абстрактным объектом, обладающим набором версий;
- объект-версия (version object) описывает различные реализации объекта-прототипа.

Image — Типы объектов: прототип / экземпляр

Вопрос 2

2.1. Понятие реляционной модели. Отношения. Ключи, их свойства и типы. Функциональная зависимость. Многозначная зависимость.

Источник: Тема 3 «Реляционная модель и нормализация».

Реляционная модель

- реляционная модель (relational database model)
  - введена в 1970г. Э.Ф.Коддом (E.F.Codd, «A Relational Model of Data for Large Shared Databanks», Communications of the ACM, 06/1970, p. 377-387);
  - основана на применении концепции реляционной алгебры (relational algebra) к проблеме хранения больших объемов данных;

- определяет способ хранения данных в виде таблиц со строками и столбцами, при этом минимизируется дублирование и исключаются определенные типы ошибок обработки;
- дает стандартный способ структурирования и обработки данных;
- фактически является промышленным стандартом хранения и обработки информации в базах данных.

Отношение

- отношение (relation) – двумерная таблица, обладающая следующими свойствами:
  - каждая строка содержит данные, описывающие некоторую сущность или ее часть;
  - каждый столбец представляет один из атрибутов сущности;
  - каждая ячейка содержит одиночное значение;
  - все значения в одном столбце имеют один и тот же тип;
  - каждый столбец имеет уникальное имя;
  - не важен порядок строк;
  - не важен порядок столбцов;
  - не может существовать двух идентичных строк.

Отношение: пример

Image — Отношение: пример

Таблица, не являющаяся отношением: пример(1)

Image — Таблица, не являющаяся отношением: пример(1)

Таблица, не являющаяся отношением: пример (2)

Image — Таблица, не являющаяся отношением: пример (2)

Альтернативная терминология

отношение (relation)	атрибут (attribute)	кортеж (tuple)
таблица (table)	столбец (column)	строка (row)
файл (file)	поле (field)	запись (record)

Ключи

- ключ (key) - один или несколько атрибутов, идентифицирующих строку отношения;
- уникальность ключа:
  - уникальный ключ (unique key) – ключ, каждое из значений которого может присутствовать в отношении не более одного раза;
  - неуникальный ключ (non-unique key) – ключ, каждое из значений которого может присутствовать в отношении более одного раза;
- составной (композитный) ключ (composite key) – ключ, состоящий из более чем одного атрибута.

Первичный ключ

- первичный ключ (primary key) – один из уникальных ключей отношения, выбранный в качестве основного для идентификации записей отношения;
  - идеальным является короткий, неизменяемый ключ числового типа;
- ключ-кандидат (candidate/alternate key) – любой из уникальных ключей отношения, не являющийся первичным;
- обозначение отношения:

ИМЯ_ОТНОШЕНИЯ(**ПЕРВИЧНЫЙ_КЛЮЧ**, список_неключевых_атрибутов)

Суррогатный ключ

- суррогатный ключ (surrogate key) – искусственно созданный уникальный ключ, используемый в качестве первичного ключа отношения;
  - как правило, создаются средствами СУБД;
  - значения суррогатных ключей бессмысленны для пользователей, соответственно, они не должны отображаться в формах и отчетах;

RENTAL_PROPERTY (**Street**, **City**, **State/Province**, **Zip/PostalCode**, **Country**, Rental_Rate)

RENTAL_PROPERTY (**PropertyID**, Street, City, State/Province, Zip/PostalCode, Country, Rental_Rate)

Выбор ключа

- уникальность определяется из требований, и не может быть определена на основании фактических значений

Image — Выбор ключа

Внешний ключ

- внешний ключ (foreign key) – первичный ключ одного отношения, помещенный в качестве атрибута в другое отношение для формирования связи между отношениями;
- очевидно, что внешний ключ, как и первичный, может быть составным;
- внешние ключи также позволяют формировать ограничения ссылочной целостности (referential integrity constraints), т.е. условие того, что внешний ключ может принимать только существующие значения соответствующего первичного ключа.

DEPARTMENT (**DepartmentName**, BudgetCode, ManagerName)

EMPLOYEE (**EmployeeNumber**, EmployeeName, *DepartmentName*)

Функциональная зависимость

- функциональная зависимость (functional dependency) имеет место, когда один из атрибутов (или несколько атрибутов) определяет значения другого атрибута (нескольких атрибутов):

```
A → B
A → (B,C) => A→B, A→C
(A,B) → C ≠> A→C, B→C
```

- левая часть функциональной зависимости называется детерминантом (determinant);

Функциональная зависимость: пример

- определение функциональных зависимостей (факта уникальности детерминанта) также производится на основе требований, а не фактических значений атрибутов

Image — Функциональная зависимость: пример

```
ObjectColor → Weight
ObjectColor → Shape
ObjectColor → (Weight, Shape)
```

Функциональная зависимость: пример (2)

Image — Функциональная зависимость: пример (2)

```
(OrderNumber, SKU) → (Quantity, Price, ExtendedPrice)
(Quantity, Price) → (ExtendedPrice)
```

Многозначные зависимости

- многозначная зависимость (multivalue dependency) имеет место, когда детерминанту соответствует некоторый набор значений;
- общая схема:

```
R(A,B,C)
A↠B
A↠C
B и C независимы
```

- возникает аномалия модификации – необходимо внести все комбинации для многозначно-зависимых атрибутов;
- очевидно, что детерминант многозначной зависимости никогда не может быть первичным ключом.

Image — Многозначные зависимости

Многозначные зависимости: пример

Image — Многозначные зависимости: пример

2.2. Реляционная модель: аномалии модификации. Функциональная зависимость. Нормализация. Первая, вторая, третья нормальные формы. Нормальная форма Бойса-Кодда.

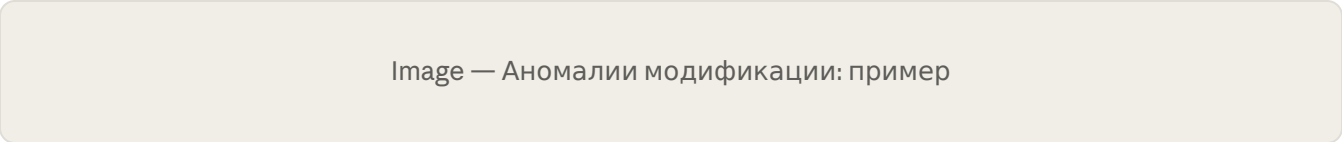
Источник: Тема 3 «Реляционная модель и нормализация».

Аномалии модификации

- аномалии модификации (modification anomalies) – нежелательные побочные эффекты, возникающие при модификации записей отношения:
  - аномалия удаления (deletion anomaly): удаление единственной строки отношения приводит к удалению информации о двух сущностях;
  - аномалия вставки (insertion anomaly): невозможность вставки информации об одной сущности до тех пор, пока не будет внесен некоторый факт о другой сущности;
  - аномалия обновления (update anomaly): некорректные результаты операции обновления.

Аномалии модификации: пример

- результаты некорректной модификации отношения EQUIPMENT_REPAIR (аномалия обновления)



Функциональная зависимость

- функциональная зависимость (functional dependency) имеет место, когда один из атрибутов (или несколько атрибутов) определяет значения другого атрибута (нескольких атрибутов):

```
A → B
A → (B,C) => A→B, A→C
(A,B) → C ≠> A→C, B→C
```

- левая часть функциональной зависимости называется детерминантом (determinant);

Понятие нормализации

- нормализация (normalization) – последовательность преобразований, обеспечивающих выполнение определенного набора требований;

Image — Понятие нормализации

Нормализация

- нормализация – процесс классификации и преобразования отношений с целью исключения аномалий модификации

Источник аномалии	Нормальные формы	Принципы проектирования
Функциональные зависимости	1НФ, 2НФ, 3НФ, БКНФ	БКНФ: каждый детерминант должен являться ключом-кандидатом
Многозначные зависимости	4НФ	4НФ: каждая многозначная зависимость должна формировать отдельное отношение
Ограничения	5НФ, ДКНФ	ДКНФ: каждое ограничение должно являться следствием ключа-кандидата или домена

Иерархия нормальных форм

- 1NF – first NF
- 2NF – second NF
- 3NF – third NF
- EKNF – Elementary Key NF
- BCNF – Boyce / Codd NF
- 4NF – fourth NF
- ETNF – Essential Tuple NF
- RFNF (KCNF) – Redundancy Free (Key Complete) NF
- SKNF – SuperKey NF
- 5NF – fifth NF
- 6NF – sixth NF

Первая нормальная форма (1НФ)

- любая таблица, удовлетворяющая определению отношения, находится в первой нормальной форме (first normal form, 1NF).

Image — Первая нормальная форма (1НФ)

Первая нормальная форма (пример)

Image — Первая нормальная форма (пример)

Вторая нормальная форма (2НФ)

- отношение находится во второй нормальной форме (second normal form, 2NF), если каждый из неключевых атрибутов зависит от всего первичного ключа;
- необходимо:
  - выделить атрибуты функциональной зависимости, нарушающей требования 2НФ, в отдельное отношение (с детерминантом качестве первичного ключа);
  - оставить детерминант в качестве внешнего ключа в исходном отношении.

Image — Вторая нормальная форма (2НФ)

Вторая нормальная форма (пример)

```
PK(bug_id, tag)
(bug_id,tag)→ tagger
tag → coiner
```

Image — Вторая нормальная форма (пример)

Третья нормальная форма (3НФ)

- отношение находится в третьей нормальной форме (third normal form, 3NF), если оно находится во второй нормальной форме и не имеет транзитивных зависимостей (transitive dependency);

- необходимо: выделить функциональную зависимость в отдельное отношение (аналогично 2НФ)

Image — Третья нормальная форма (3НФ)

Третья нормальная форма (пример)

Image — Третья нормальная форма (пример)

Нормальная форма Бойса-Кодда (БКНФ)

- отношение находится в нормальной форме Бойса-Кодда (Boyce-Codd normal form, BC/NF), если каждый детерминант является ключом-кандидатом.

Image — Нормальная форма Бойса-Кодда (БКНФ)

> “I swear to construct my tables so that all non-key columns are dependent on the key, the whole key and nothing but the key, so help me Codd.”

Нормальная форма Бойса-Кодда (пример)

```
PK(bug_id, tag)
AK(bug_id, tag_type)
(tag)→ tag_type
```

Аномалии: вставка, изменение, удаление

Image — Нормальная форма Бойса-Кодда (пример)

БКНФ: процедура непосредственного приведения

1. выделить все функциональные зависимости;
2. определить все ключи-кандидаты;
3. если существует функциональная зависимость, детерминант которой не является ключом-кандидатом:

- необходимо выделить атрибуты, участвующие в данной функциональной зависимости в новое отношение;
- в качестве первичного ключа нового отношения будет выступать детерминант функциональной зависимости;
- детерминант функциональной зависимости также должен остаться в исходном отношении в качестве внешнего ключа (с заданием ограничения ссылочной целостности);

4. шаги могут повторяться многократно до тех пор, пока для каждого из отношений не выполняются требования БКНФ.

БКНФ: пример – исходное отношение

Image — БКНФ: пример – исходное отношение

EQUIPMENT_REPAIR (ItemNumber, Type, AcquisitionCost, **RepairNumber**, RepairDate, RepairAmount)

```
ItemNumber → (Type, AcquisitionCost)
RepairNumber → (ItemNumber, Type, AcquisitionCost, RepairDate, RepairCost)
```

БКНФ: пример – отношения в БКНФ

Image — БКНФ: пример – отношения в БКНФ

ITEM (**ItemNumber**, Type, AcquisitionCost)

REPAIR (*ItemNumber*, **RepairNumber**, RepairDate, RepairAmount)

Where REPAIR.ItemNumber must exist in ITEM.ItemNumber

2.3. Реляционная модель: аномалии модификации. Многозначная зависимость. Нормализация. Четвертая нормальная форма. Пятая нормальная форма. Нормализация и денормализация.

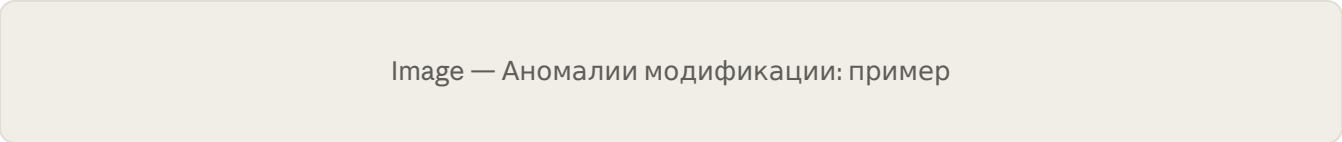
Источник: Тема 3 «Реляционная модель и нормализация».

Аномалии модификации

- аномалии модификации (modification anomalies) – нежелательные побочные эффекты, возникающие при модификации записей отношения:
  - аномалия удаления (deletion anomaly): удаление единственной строки отношения приводит к удалению информации о двух сущностях;
  - аномалия вставки (insertion anomaly): невозможность вставки информации об одной сущности до тех пор, пока не будет внесен некоторый факт о другой сущности;
  - аномалия обновления (update anomaly): некорректные результаты операции обновления.

Аномалии модификации: пример

- результаты некорректной модификации отношения EQUIPMENT_REPAIR (аномалия обновления)

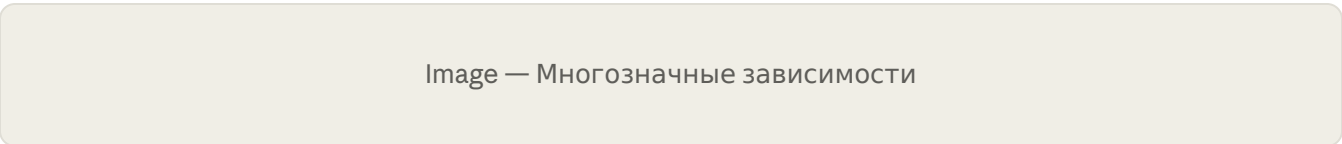


Многозначные зависимости

- многозначная зависимость (multivalue dependency) имеет место, когда детерминанту соответствует некоторый набор значений;
- общая схема:

```
R(A,B,C)
A↠B
A↠C
B и C независимы
```

- возникает аномалия модификации – необходимо внести все комбинации для многозначно-зависимых атрибутов;
- очевидно, что детерминант многозначной зависимости никогда не может быть первичным ключом.



Многозначные зависимости: пример

Image — Многозначные зависимости: пример

Понятие нормализации

- нормализация (normalization) – последовательность преобразований, обеспечивающих выполнение определенного набора требований;

Image — Понятие нормализации

Нормализация

- нормализация – процесс классификации и преобразования отношений с целью исключения аномалий модификации

Источник аномалии	Нормальные формы	Принципы проектирования
Функциональные зависимости	1НФ, 2НФ, 3НФ, БКНФ	БКНФ: каждый детерминант должен являться ключом-кандидатом
Многозначные зависимости	4НФ	4НФ: каждая многозначная зависимость должна формировать отдельное отношение
Ограничения	5НФ, ДКНФ	ДКНФ: каждое ограничение должно являться следствием ключа-кандидата или домена

Четвертая нормальная форма (4НФ)

- отношение находится в четвертой нормальной форме (fourth normal form, 4NF), если оно находится в нормальной форме Бойса-Кодда и не имеет нетривиальных многозначных зависимостей (все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от его ключей).

- или… : отношение находится в четвёртой нормальной форме тогда и только тогда, когда в случае существования таких подмножеств A и B атрибутов этого отношения, для которых выполняется нетривиальная многозначная зависимость A →→ B, все атрибуты переменной отношения R также функционально зависят от A.

Image — Четвертая нормальная форма (4НФ)

Четвертая нормальная форма (пример)

- проблема возникает, если:
  - существуют атрибуты, не участвующие в многозначной зависимости;
  - существуют несколько *независимых* многозначных зависимостей.

Image — Четвертая нормальная форма (пример)

Пятая нормальная форма

- Fifth Normal Form (5NF) / Project-Join Normal Form (PJ/NF);
- рассматривает случаи, когда информация может быть восстановлена из отдельных составляющих, обладающих меньшей избыточностью;
- в случае отсутствия дополнительных семантических правил четвертая и пятая нормальные формы эквивалентны;
- при условии отсутствия составного первичного ключа и выполнении условий БКНФ, отношение автоматически находится в 5НФ.

Пятая нормальная форма: пример

Image — Пятая нормальная форма: пример

- правило: если агент представляет компанию и продает продукты определенного типа, то он продает продукты данного типа всех компаний, которые он представляет

Доменно-ключевая нормальная форма (ДКНФ)

- R.Fagin, «A Normal Form for Relational Databases That Is Based On Domains and Keys», ACM Transactions on Database Systems, September 1981, pp. 387-415
- отношение находится в доменно-ключевой нормальной форме (domain-key normal form, DKNF), если каждое из наложенных на него ограничений является логическим следствием определения доменов и ключей;
- ограничение – любое правило, регулирующее возможные статические значения атрибутов и достаточно точное, чтобы можно было установить его истинность или ложность:
  - правила редактирования;
  - ограничения связей;
  - внутренние ограничения отношений;
  - функциональные зависимости;
  - многозначные зависимости;
- к указанным ограничениям не относятся:
  - ограничения изменений значений данных;
  - ограничения, зависящие от времени;
- домен (domain) – именованное множество допустимых значений атрибута.

Нормализация и денормализация

- преимущества нормализации:
  - отсутствие аномалий модификации;
  - минимизация избыточности данных:
    - более простое поддержание целостности данных;
    - меньшие накладные расходы на хранение;
- недостатки нормализации:
  - более сложные запросы для извлечения данных;
  - следовательно, большие накладные расходы на их выполнение, что может снижать производительность;
- тип базы данных:
  - обновляемая («updateable», operational): OLTP (OnLine Transaction Processing);
  - преимущественно не обновляемая («read mostly», non-operational): OLAP (OnLine Analytic Processing)

Нормализация и денормализация: пример

Image — Нормализация и денормализация: пример

2.4. Преобразование модели «сущность-связь»: основные этапы. Создание таблиц для каждой из сущностей. Определение свойств столбцов таблицы. Нормализация.

Источник: Тема 4 «Преобразование модели «сущность-связь» в реляционную модель»; Тема 3 «Реляционная модель и нормализация».

Основные шаги

1. создание таблицы для каждой сущности:
   - определение первичного ключа (возможно, суррогатного);
   - определение ключей-кандидатов;
   - определение свойств каждого столбца: типа данных (data type), возможности неопределенного значения (null value), значения по умолчанию (default value), ограничений на значения (data constraints);
   - проверка нормализации (как правило, нормализация производится до БКНФ/4НФ);
2. создание связей с помощью внешних ключей:
   - между сильными сущностями (1:1, 1:N, N:M);
   - для идентификационно-зависимых сущностей;
   - для слабых сущностей;
   - для сущностей тип-подтип;
   - рекурсивных;
3. обеспечение условий минимальной кардинальности

Создание отношения для сущности и выбор первичного ключа

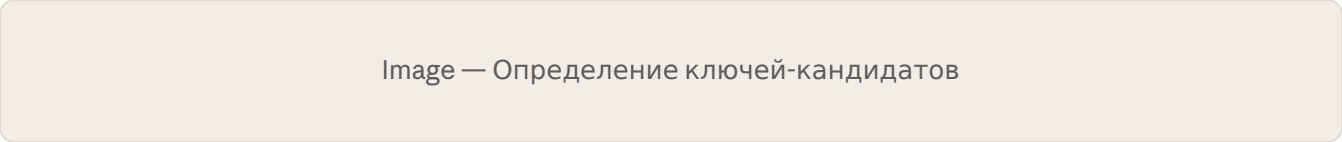
EMPLOYEE (**EmployeeNumber**, EmployeeName, Phone, Email, HireDate, ReviewDate, EmpCode)

Image — Создание отношения для сущности и выбор первичного ключа

- идеальным является короткий, неизменяемый ключ числового типа;

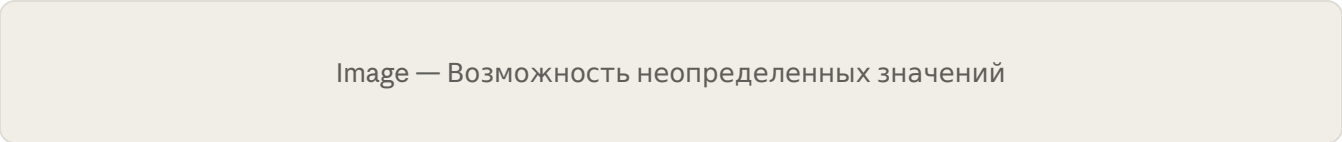
- суррогатный ключ (surrogate key) – искусственно созданный уникальный ключ, используемый в качестве первичного ключа (primary key).

Определение ключей-кандидатов



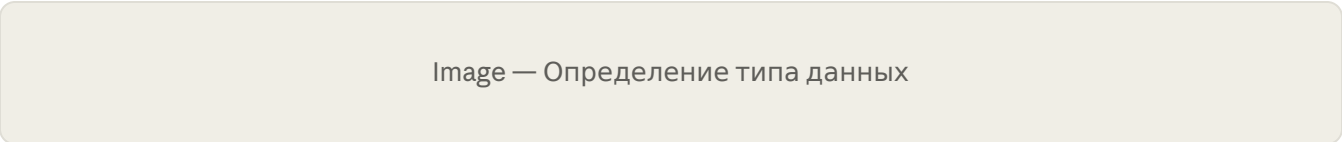
- ключ-кандидат (candidate/alternate key) – любой из уникальных ключей отношения, не являющийся первичным;
- для обозначения можно использовать обозначения ERwin: AKn.m
  - n – номер ключа-кандидата;
  - m – номер атрибута в соответствующем ключе-кандидате.

Возможность неопределенных значений

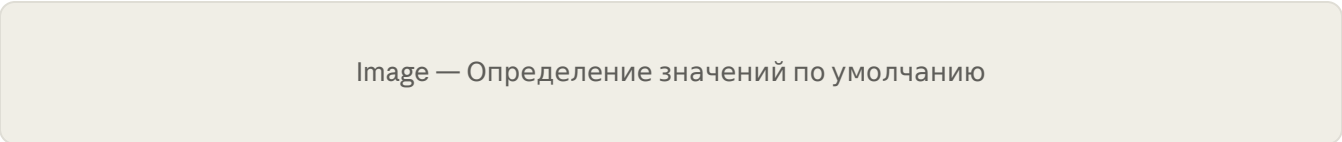


- отсутствующие значения (missing values, null values):
  - неприменимое значение (inappropriate value);
  - неизвестное значение (unknown value);
  - не введенное значение.

Определение типа данных



Определение значений по умолчанию



- значение по умолчанию (default value) – автоматически создаваемое СУБД значение при вставке новой записи, если оно не задано явно.

Определение ограничений

- категории целостности данных:

    - *сущностная*: определяет строку как уникальную сущность в конкретной таблице, поддерживается через введение ключа;
    - *доменная*: определяет достоверность записей в конкретном столбце;
    - *ссылочная*: сохраняет определенные связи между таблицами при вводе или удалении записей;
    - *пользовательская* целостность;
- ограничения:
    - *домена* (domain constraint): значения атрибута должны быть из определенного множества;
    - *интервальные* (range constraint): значения атрибута должны быть в определенном интервале;
    - *внутренние* (intrarelation constraint): значения атрибута должны удовлетворять определенным условиям сравнения с другими атрибутами того же отношения;
    - *внешние* (interrelation constraint): значения атрибута должны удовлетворять определенным условиям сравнения с атрибутами другого отношения – ограничения ссылочной целостности (referential integrity constraint) посредством внешних ключей.

Нормализация

- нормализация – процесс классификации и преобразования отношений с целью исключения аномалий модификации

Источник аномалии	Нормальные формы	Принципы проектирования
Функциональные зависимости	1НФ, 2НФ, 3НФ, БКНФ	БКНФ: каждый детерминант должен являться ключом-кандидатом
Многозначные зависимости	4НФ	4НФ: каждая многозначная зависимость должна формировать отдельное отношение
Ограничения	5НФ, ДКНФ	ДКНФ: каждое ограничение должно являться следствием ключа-кандидата или домена

2.5. Преобразование модели «сущность-связь»: основные этапы. Создание связей: сильные сущности, идентификационно-зависимые сущности, шаблоны «сопряжение» и «прототип-экземпляр».

Источник: Тема 4 «Преобразование модели «сущность-связь» в реляционную модель».

Основные шаги

1. создание таблицы для каждой сущности:
   - определение первичного ключа (возможно, суррогатного);
   - определение ключей-кандидатов;
   - определение свойств каждого столбца: типа данных (data type), возможности неопределенного значения (null value), значения по умолчанию (default value), ограничений на значения (data constraints);
   - проверка нормализации (как правило, нормализация производится до БКНФ/4НФ);
2. создание связей с помощью внешних ключей:
   - между сильными сущностями (1:1, 1:N, N:M);
   - для идентификационно-зависимых сущностей;
   - для слабых сущностей;
   - для сущностей тип-подтип;

- рекурсивных;

3. обеспечение условий минимальной кардинальности

Создание связей: сильные сущности 1:1

Image — Создание связей: сильные сущности 1:1

- первичный ключ отношения, соответствующего одной из сущностей, помещается в качестве внешнего ключа в отношение, представляющее вторую сущность.

Создание связей: сильные сущности 1:N

Image — Создание связей: сильные сущности 1:N

- первичный ключ отношения, соответствующего родительской сущности, помещается в качестве внешнего ключа в отношение, представляющее дочернюю сущность.

Создание связей: сильные сущности M:N

Image — Создание связей: сильные сущности M:N

COMPANY_PART_INT (**CompanyName**, **PartNumber**)

- создается промежуточное отношение (intersection table):
  - содержит в качестве внешних первичные ключи обоих отношений;
  - оба внешних ключа формируют составной первичный ключ нового отношения;
  - другие поля отсутствуют.

Создание связей: идентификационно-зависимые сущности

- идентификационно-зависимой (ID-dependent) называется такая дочерняя сущность, идентификатор которой содержит идентификатор родительской сущности (дочерняя сущность является логическим расширением или составной частью родительской);
- варианты использования идентификационно-зависимых сущностей:

- шаблон «сопряжение» (association pattern);
- многозначные атрибуты (multivalued attributes);
- шаблон «прототип-экземпляр» (archetype/instance pattern);

Image — Создание связей: идентификационно-зависимые сущности

Создание связей: шаблон «сопряжение»

Image — Создание связей: шаблон «сопряжение»

QUOTATION (**CompanyName**, **PartNumber**, Price)

- создается промежуточное отношение (association table):
  - содержит в качестве внешних первичные ключи обоих отношений;
  - оба внешних ключа формируют составной первичный ключ нового отношения;
  - присутствуют дополнительные атрибуты.

Создание связей: шаблон «прототип-экземпляр»

Image — Создание связей: шаблон «прототип-экземпляр»

Создание связей: шаблон «прототип-экземпляр» - слабые сущности

Image — Создание связей: шаблон «прототип-экземпляр» - слабые сущности

- слабой сущностью (weak entity) называется сущность, существование которой зависит от наличия родительской сущности;
- все идентификационно-зависимые сущности являются слабыми;
- возможно наличие в модели слабых сущностей, которые при этом не являются идентификационно-зависимыми;

Создание связей: сложный пример (1)

Image — Создание связей: сложный пример (1)

Создание связей: сложный пример (2)

Image — Создание связей: сложный пример (2)

2.6. Преобразование модели «сущность-связь»: основные этапы. Создание связей: шаблон «категория-подтип», представление многозначных атрибутов. Рекурсивные связи.

Источник: Тема 4 «Преобразование модели «сущность-связь» в реляционную модель».

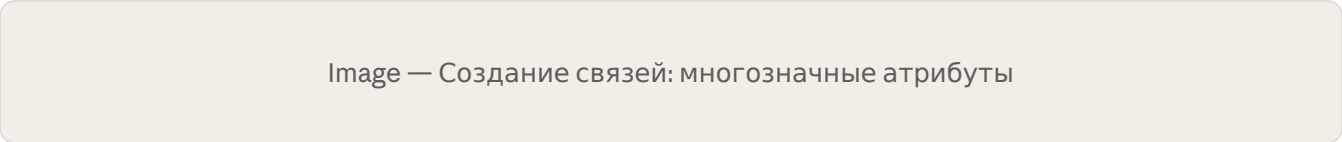
Основные шаги

1. создание таблицы для каждой сущности:
   - определение первичного ключа (возможно, суррогатного);
   - определение ключей-кандидатов;
   - определение свойств каждого столбца: типа данных (data type), возможности неопределенного значения (null value), значения по умолчанию (default value), ограничений на значения (data constraints);
   - проверка нормализации (как правило, нормализация производится до БКНФ/4НФ);
2. создание связей с помощью внешних ключей:
   - между сильными сущностями (1:1, 1:N, N:M);
   - для идентификационно-зависимых сущностей;
   - для слабых сущностей;
   - для сущностей тип-подтип;
   - рекурсивных;
3. обеспечение условий минимальной кардинальности

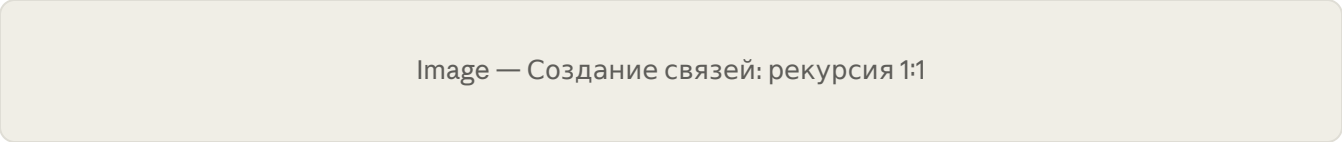
Создание связей: подтипы

Image — Создание связей: подтипы

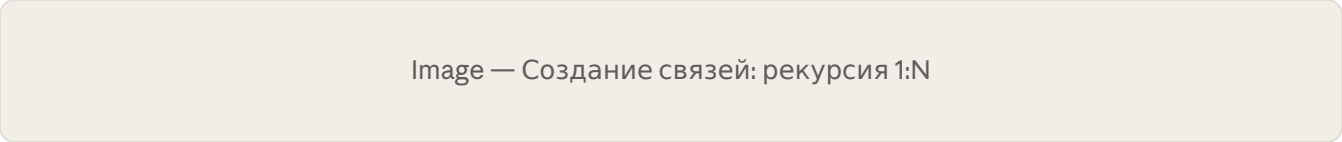
Создание связей: многозначные атрибуты



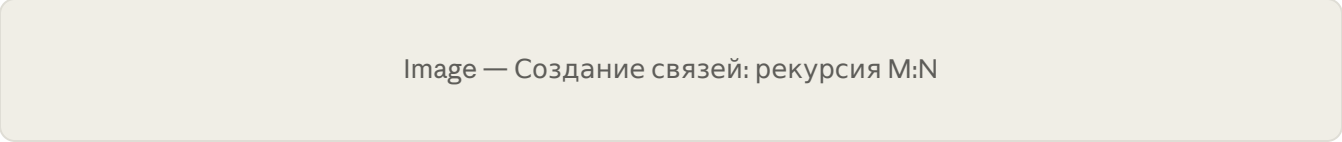
Создание связей: рекурсия 1:1



Создание связей: рекурсия 1:N



Создание связей: рекурсия M:N



2.7. Преобразование модели «сущность-связь»: основные этапы. Создание связей: ограничения минимальной кардинальности связей.

Источник: Тема 4 «Преобразование модели «сущность-связь» в реляционную модель».

Основные шаги

1. создание таблицы для каждой сущности:
   - определение первичного ключа (возможно, суррогатного);
   - определение ключей-кандидатов;
   - определение свойств каждого столбца: типа данных (data type), возможности неопределенного значения (null value), значения по умолчанию (default value), ограничений на значения (data constraints);
   - проверка нормализации (как правило, нормализация производится до БКНФ/4НФ);
2. создание связей с помощью внешних ключей:
   - между сильными сущностями (1:1, 1:N, N:M);

    - для идентификационно-зависимых сущностей;
    - для слабых сущностей;
    - для сущностей тип-подтип;
    - рекурсивных;
3. обеспечение условий минимальной кардинальности

Ограничения минимальной кардинальности

- связи могут иметь следующие варианты минимальной кардинальности:
    - O-O (optional-optional): родительская и дочерняя сущности являются необязательными;
    - M-O (mandatory-optional): родительская сущность является обязательной, дочерняя сущность является необязательной;
    - O-M (optional-mandatory): родительская сущность является необязательной, дочерняя сущность является обязательной;
    - M-M (mandatory-mandatory): родительская и дочерняя сущности являются обязательными;
- часто для обозначения процедуры обеспечения ограничений минимальной кардинальности используют термин action (действие);
- рассматриваемые операции над записями:
    - вставка (insert);
    - изменение (update) первичного или внешнего ключа;
    - удаление (delete);
- очевидно, что обеспечение ограничения O-O не требует никаких действий.

Каскадные ограничения ссылочной целостности

- каскадное обновление (cascade update) имеет место, когда необходимо изменить значение(я) внешнего ключа (в дочерней таблице) при изменении первичного ключа родительской:
    - заметим, что суррогатные ключи не изменяются, следовательно, для них не нужно каскадное обновление;
- каскадное удаление (cascade delete) имеет место, когда необходимо удалить строки в дочерней таблице при удалении записи в родительской таблице:
    - для сильных сущностей: как правило, не используется;
    - для слабых сущностей: как правило, используется.

Image — Каскадные ограничения ссылочной целостности

Действия, необходимые для связей с минимальной кардинальностью M-x

Операция	Действие над родительской таблицей	Действие над дочерней таблицей
Вставка	---	• подбор новой родительской записи; • запрет.
Изменение	• каскадное обновление; • запрет.	• допустимо, если новое значение внешнего ключа соответствует некоторому первичному в родительской таблице; • запрет.
Удаление	• каскадное удаление; • запрет.	---

Действия, необходимые для связей с минимальной кардинальностью x-M

Операция	Действие над родительской таблицей	Действие над дочерней таблицей
Вставка	• подбор новой дочерней записи; • запрет.	---
Изменение	• обновление внешнего ключа хотя бы одной дочерней записи; • запрет.	• если есть другие дочерние записи у соответствующей родительской, доп. действия не нужны; • иначе, запрет или поиск замены.
Удаление	---	• если есть другие дочерние записи у соответствующей родительской, доп. действия не нужны; • иначе, запрет или поиск замены.

Действия, необходимые для обеспечения ограничений минимальной кардинальности

O-O	---	---
M-O	Действия, необходимые для связей с минимальной кардинальностью M-x	обеспечивается средствами СУБД: • определение ограничений ссылочной целостности (PK+FK); • задание условия NOT NULL для внешнего ключа.
O-M	Действия, необходимые для связей с минимальной кардинальностью x-M	• обеспечивается средствами СУБД путем создания триггеров или использования других средств программирования; • обеспечивается внешними по отношению к СУБД средствами.
M-M	Оба набора действий (M-x + x-M)	• обеспечивается средствами СУБД или внешними по отношению к СУБД средствами; • обеспечение ограничений минимальной кардинальности является в данном случае крайне сложной задачей (особенно для сильных сущностей).

- триггер – некоторый хранимый программный код, который выполняется при наступлении некоторого события

Действия для связей M-O: пример

- действия для DEPARTMENT:
  - вставка: без ограничений;
  - изменение первичного ключа: каскадное обновление;
  - удаление:
    - каскадное удаление;
    - запрет на удаление;
- действия для EMPLOYEE (подразумевается определение ограничений ссылочной целостности (PK+FK) и задание условия NOT NULL для внешнего ключа):
  - вставка: возможна только при корректном значении внешнего ключа;

- изменение внешнего ключа: возможно только при корректном новом значении;
- удаление: без ограничений.

Image — Действия для связей M-O: пример

Действия для связей O-M: пример

- действия для DEPARTMENT:
  - вставка: операция возможна только при одновременном создании или выборе записи в дочерней таблице: необходим триггер;
  - изменение первичного ключа: операция возможна только при одновременном выборе записи (изменении внешнего ключа в этой записи) в дочерней таблице: необходим триггер;
  - удаление: без ограничений;
- действия для EMPLOYEE:
  - вставка: без ограничений;
  - изменение внешнего ключа: операция возможна, только если к родительской записи относится более одной дочерней записи;
  - удаление: операция возможна, только если к родительской записи относится более одной дочерней записи.

Image — Действия для связей O-M: пример

Пример: ограничения минимальной кардинальности

Image — Пример: ограничения минимальной кардинальности

Пример: ограничения минимальной кардинальности, ARTIST-to-WORK, M-O

Image — Пример: ограничения минимальной кардинальности, ARTIST-to-WORK, M-O

Пример: ограничения минимальной кардинальности, WORK-to-TRANS, M-O

Image — Пример: ограничения минимальной кардинальности, WORK-to-TRANS, M-O

Пример: ограничения минимальной кардинальности, WORK-to-TRANS, O-M

Image — Пример: ограничения минимальной кардинальности, WORK-to-TRANS, O-M

Пример: ограничения минимальной кардинальности, CUSTOMER-to-INT, M-O

Image — Пример: ограничения минимальной кардинальности, CUSTOMER-to-INT, M-O

Пример: ограничения минимальной кардинальности, ARTIST-to-INT, M-O

Image — Пример: ограничения минимальной кардинальности, ARTIST-to-INT, M-O

2.8. Преобразование модели семантических объектов. Типы объектов: простой, составной, сложный.

Источник: Тема 4 «Преобразование модели семантических объектов в реляционную модель».

Типы семантических объектов

- простой объект;
- составной объект:
  - с независимыми группами;
  - с вложенными группами;
- сложный объект:
  - 1:1;
  - 1:N;
  - M:N;
- гибридный объект;
- ассоциативный объект;

- родитель / подтип;
- прототип / экземпляр.

Простой объект

- простой объект (simple object) – объект, содержащий только однозначные простые или групповые атрибуты

Image — Простой объект

Составной объект с независимыми группами

- составной объект (composite object) – объект, содержащий один или более многозначный простой или групповой атрибуты

Image — Составной объект с независимыми группами

Составной объект с вложенными группами

Image — Составной объект с вложенными группами

Сложный объект (1:1)

- сложный объект (compound object) – объект, содержащий один или более объектный атрибут

Image — Сложный объект (1:1)

Сложный объект (1:N)

Image — Сложный объект (1:N)

Сложный объект (M:N)

Image — Сложный объект (M:N)

2.9. Преобразование модели семантических объектов. Типы объектов: гибридный, ассоциативный, «родитель-подтип», «прототип-экземпляр».

Источник: Тема 4 «Преобразование модели семантических объектов в реляционную модель».

Типы семантических объектов

- простой объект;
- составной объект:
  - с независимыми группами;
  - с вложенными группами;
- сложный объект:
  - 1:1;
  - 1:N;
  - M:N;
- гибридный объект;
- ассоциативный объект;
- родитель / подтип;
- прототип / экземпляр.

Гибридный объект: варианты кардинальности

Image — Гибридный объект: варианты кардинальности

Гибридный объект: схема преобразования

Image — Гибридный объект: схема преобразования

Гибридный объект: пример

- гибридный объект (hybrid object) – объект, содержащий один или более групповой атрибут, содержащий в себе объектный атрибут

Image — Гибридный объект: пример

Ассоциативный объект

- ассоциативный объект (association object) – объект, сопоставляющий два или более объектов, и содержащий дополнительную информацию о таком сопоставлении

Image — Ассоциативный объект

Родитель / подтип

Image — Родитель / подтип

Прототип / экземпляр

Image — Прототип / экземпляр

Вопрос 3

3.1. Ядро СУБД. Язык SQL: стандарты, категории операторов.

Источник: Тема 1 «Введение»; Тема 8 «SQL. Операторы DML».

Компоненты современных СУБД

- ядро СУБД (database engine) обеспечивает хранение, обработку и защиту данных:
  - преобразование запросов в действия над данными;
  - безопасность;
  - управление транзакциями и блокировками;
  - резервное копирование и восстановление;
- средства интерактивной аналитической обработки (OLAP) и средства интеллектуального анализа данных;

- средства интеграции данных (экспорт и импорт данных);
- средства создания и публикации форм и отчетов, создания форм;
- средства репликации (копирования и распространения данных и объектов баз данных из одной базы данных в другую с возможностью последующей синхронизации между базами данных для поддержания согласованности);
- средства выполнения полнотекстовых запросов к неструктурированным символьным данным в таблицах;
- …

Image — Компоненты современных СУБД

Язык SQL

- язык SQL (Structured Query Language - «язык структурированных запросов») – язык манипулирования реляционными данными
- язык SQL представляет собой совокупность
  - операторов;
  - инструкций;
  - вычисляемых функций;

Image — Язык SQL

Стандарты языка SQL

- язык был разработан в конце 1970-х в IBM Corporation;
- был принят в качестве национального стандарта (American National Standards Institute, ANSI) в 1992 году – известен как **SQL-92**.
- новые версии стандарта включают в себя элементы XML и объектно-ориентированных концепций:
  - SQL:1999;
  - SQL:2003;
  - SQL:2006;
  - SQL:2008.

Категории операторов SQL

- операторы определения данных (Data Definition Language, DDL):
  - CREATE;
  - ALTER;
  - DROP;
- операторы манипуляции данными (Data Manipulation Language, DML):
  - SELECT;
  - INSERT;
  - UPDATE;
  - DELETE;
- операторы определения доступа к данным (Data Control Language, DCL):
  - GRANT;
  - REVOKE;
  - DENY;
- операторы управления транзакциями (Transaction Control Language, TCL):
  - COMMIT;
  - ROLLBACK.

3.2. Выборка данных: структура оператора SELECT. Указание списка выбора и источника. Использование псевдонимов.

Источник: Тема 8 «SQL. Операторы DML».

Выборка данных: SELECT (SELECT statement)

- FROM – источник выборки;
- WHERE – условие выборки;
- HAVING – условие группировки;
- GROUP BY – список группировки;
- ORDER BY – сортировка результатов;
- UNION, EXCEPT, INTERCEPT – комбинирование результатов нескольких выборок.

Пример базы данных

Image — Пример базы данных

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
2	AVTORIF	NULL
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

SELECT: список выбора и указание источника

- список выбора:

    - список всех полей;
    - отдельные поля;
    - определение порядка отдельных полей;
    - псевдонимы для полей;
    - использование выражений и выражений с псевдонимами;
- источник (FROM…):
    - единственная таблица;
    - несколько таблиц;
    - псевдонимы для таблиц;
- фильтрация уникальных строк: DISTINCT.

SELECT: указание списка полей

```
SELECT [PartID],[PartNumber],[Description]
FROM [Parts]
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

SELECT: указание списка полей (2)

```
SELECT [PartNumber],[Description]
FROM [Parts]
```

PartNumber	Description
MZ311785	Фильтр воздушный
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный
MB891677	Амортизатор задний
MR389546	Колодки передние
MZ690170	Колодки задние

SELECT: **указание списка полей** (3)

```
SELECT [Description], [PartID],[PartNumber]
FROM [Parts]
```

Description	PartID	PartNumber
Фильтр воздушный	1	MZ311785
Фильтр масляный	2	ME013343
Фильтр масляный	3	1230A046
Фильтр масляный	4	ME215002
Фильтр топливный	6	MB220900
Фильтр топливный	7	XB220900
Амортизатор задний	8	MB891677
Колодки передние	9	MR389546
Колодки задние	10	MZ690170

SELECT: **указание списка полей** (4)

```
SELECT *
FROM [Parts]
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

SELECT: псевдонимы полей

```
SELECT [PartID] as [Identity], [PartNumber],[Description]
FROM [Parts]
```

Identity	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние

Identity	PartNumber	Description
10	MZ690170	Колодки задние

SELECT: псевдонимы таблиц

```
SELECT a.[PartNumber], a.[Description], c.[SupplierCode], b.[Price],
  b.[DeliveryTime]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID]
```

PartNumber	Description	SupplierCode	Price	DeliveryTime
ME013343	Фильтр масляный	UAE	55	2
ME215002	Фильтр масляный	AVTORIF	200	35
ME215002	Фильтр масляный	UAE	100	5
ME215002	Фильтр масляный	GER	200	60

SELECT: уникальность строк в выборке

```
SELECT a.[PartNumber], a.[Description]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный

```
SELECT DISTINCT a.[PartNumber], a.[Description]
FROM [Parts] as a, [Supply] as b, [Suppliers] as c
WHERE a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

3.3. Выборка данных: условие отбора записей и сортировка. Значение NULL и трехзначная логика.

Источник: Тема 8 «SQL. Операторы DML».

SELECT: условие выбора записей и сортировка

- условие выбора записей (WHERE…) – задается с помощью логического выражения, формируемого с помощью:
  - операторов сравнения (>, <, =, <>, <=, >=);
  - логических операторов (AND, OR, NOT);
  - задания диапазона (*value* BETWEEN *min* AND *max*);
  - условия принадлежности множеству значений (*value* IN (*list*), список может формироваться с помощью вложенных запросов);
  - условия непустого множества строк, возвращаемого в результате выполнения подзапроса (EXISTS (subquery));
  - задания шаблона для строковых значений (value LIKE '…')
    - символ % является шаблоном подстроки;
    - символ _ является шаблоном единственного символа;
    - [],[^] позволяют указать/ограничить допустмый набор/диапазон символов;
  - определения наличия неопределенного значения (IS NULL | IS NOT NULL)
- сортировка (ORDER BY):
  - ASC – по возрастанию (по умолчанию);
  - DESC – по убыванию.

SELECT: сортировка строк в выборке

```
SELECT a.[PartNumber],
  a.[Description]
FROM [Parts] as a
```

PartNumber	Description
MZ311785	Фильтр воздушный
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный
MB891677	Амортизатор задний
MR389546	Колодки передние
MZ690170	Колодки задние

```
SELECT a.[PartNumber],
  a.[Description]
FROM [Parts] as a
ORDER BY a.[PartNumber]
```

PartNumber	Description
1230A046	Фильтр масляный
MB220900	Фильтр топливный
MB891677	Амортизатор задний
ME013343	Фильтр масляный
ME215002	Фильтр масляный
MR389546	Колодки передние
MZ311785	Фильтр воздушный
MZ690170	Колодки задние
XB220900	Фильтр топливный

SELECT: порядок сортировки

```
SELECT a.[PartNumber], a.[Description]
FROM [Parts] as a
ORDER BY a.[PartNumber] asc
```

PartNumber	Description
1230A046	Фильтр масляный
MB220900	Фильтр топливный
MB891677	Амортизатор задний
ME013343	Фильтр масляный
ME215002	Фильтр масляный
MR389546	Колодки передние
MZ311785	Фильтр воздушный
MZ690170	Колодки задние
XB220900	Фильтр топливный

```
SELECT a.[PartNumber], a.[Description]
FROM [Parts] as a
ORDER BY a.[PartNumber] desc
```

PartNumber	Description
XB220900	Фильтр топливный
MZ690170	Колодки задние
MZ311785	Фильтр воздушный
MR389546	Колодки передние
ME215002	Фильтр масляный
ME013343	Фильтр масляный
MB891677	Амортизатор задний
MB220900	Фильтр топливный

PartNumber	Description
1230A046	Фильтр масляный

SELECT: условия выбора строк (between)

```
SELECT a.[PartNumber], a.[Description], c.[SupplierCode], b.[Price], b.
[DeliveryTime]
 FROM [Parts] as a, [Supply] as b, [Suppliers] as c
 WHERE (a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID])
```

PartNumber	Description	SupplierCode	Price	DeliveryTime
ME013343	Фильтр масляный	UAE	55	2
ME215002	Фильтр масляный	AVTORIF	200	35
ME215002	Фильтр масляный	UAE	100	5
ME215002	Фильтр масляный	GER	200	60

```
SELECT a.[PartNumber], a.[Description], c.[SupplierCode], b.[Price], b.
[DeliveryTime]
 FROM [Parts] as a, [Supply] as b, [Suppliers] as c
 WHERE (a.[PartID] = b.[PartID] and b.[SupplierID] = c.[SupplierID])
 and b.Price between 12 and 100
```

PartNumber	Description	SupplierCode	Price	DeliveryTime
ME013343	Фильтр масляный	UAE	55	2
ME215002	Фильтр масляный	UAE	100	5

SELECT: условия выбора строк (in)

```
SELECT a.[PartID], a.[PartNumber],
   a.[Description]
FROM [Parts] as a
```

PartID	PartNumber	Description
1	MZ311785	Фильтр воздушный
2	ME013343	Фильтр масляный
3	1230A046	Фильтр масляный
4	ME215002	Фильтр масляный
6	MB220900	Фильтр топливный
7	XB220900	Фильтр топливный
8	MB891677	Амортизатор задний
9	MR389546	Колодки передние
10	MZ690170	Колодки задние

```
SELECT * FROM [Supply]
```

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

```
SELECT a.[PartNumber],
   a.[Description]
FROM [Parts] as a
WHERE a.[PartID] in (select PartID
   from [Supply])
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: условия выбора строк (exists)

```
SELECT a.[PartNumber],
   a.[Description]
FROM [Parts] as a
WHERE exists (select PartID from
   [Supply] as b where a.[PartID] =
   b.[PartID])
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: подзапрос со сложным условием

```
SELECT a.[PartNumber], a.[Description]
FROM [Parts] as a
WHERE exists (
   select PartID
   from [Parts] as b
   where (
      (a.[PartID] <> b.[PartID]) and
      (a.[Description] = b.[Description])
   ))
```

PartNumber	Description
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный

PartNumber	Description
XB220900	Фильтр топливный

SELECT: условия выбора строк (like)

```
SELECT a.[PartNumber],
  a.[Description]
FROM [Parts] as a
WHERE
  a.[PartNumber] like '%0%'
```

PartNumber	Description
ME013343	Фильтр масляный
1230A046	Фильтр масляный
ME215002	Фильтр масляный
MB220900	Фильтр топливный
XB220900	Фильтр топливный
MZ690170	Колодки задние

SELECT: условия выбора строк (функции)

```
SELECT * FROM [Suppliers]
WHERE LEN(SupplierCode) = 3
```

SupplierID	SupplierCode	Name
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

SELECT: неопределенные значения (NULL)

```
SELECT *
FROM suppliers
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
2	AVTORIF	NULL
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

```
set ansi_nulls off
SELECT * from suppliers
where Name = null
```

SupplierID	SupplierCode	Name
2	AVTORIF	NULL

```
SELECT * from suppliers
where Name <> null
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

```
set ansi_nulls on
SELECT * from suppliers
where Name = null
```

SupplierID	SupplierCode	Name

```
SELECT * from suppliers
where Name <> null
```

SupplierID	SupplierCode	Name

Значения NULL и трехзначная логика (3VL, three-value logic)

Image — Значения NULL и трехзначная логика (3VL, three-value logic)

SELECT: **неопределенные значения** (NULL)

```
SELECT *
FROM suppliers
WHERE Name IS NULL
```

SupplierID	SupplierCode	Name
2	AVTORIF	NULL

```
SELECT *
FROM suppliers
WHERE Name IS NOT NULL
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

3.4. Выборка данных: группировка, условия группировки и функции агрегирования.

Источник: Тема 8 «SQL. Операторы DML».

SELECT: группировка, условия группировки и функции агрегирования

- список полей, по которым осуществляется группировка:

`GROUP BY …`

- HAVING позволяет задавать условия с функциями агрегирования - данное условие выполняется после отбора записей по условию WHERE;
- функции агрегирования: COUNT, MIN, MAX, SUM, AVG
  - все поля из списка выборки, не задействованные в функциях агрегирования, должны присутствовать в списке GROUP BY;
  - с помощью COUNT можно подсчитывать количество строк в таблице (COUNT(*)) или значений в столбце

SELECT: подсчет строк и значений (count)

```
select count(*)
from parts
```

(No column name)
9

```
select count(PartID) as cnt
from parts
```

cnt
9

SELECT: подсчет строк и значений (2)

```
select count(*)
from suppliers
```

(No column name)
5

```
select count(Name)
from suppliers
```

(No column name)
4

SELECT: подсчет уникальных значений

```
select
   count(Description)
from parts
```

(No column name)
9

```
select
   count(distinct
   Description)
from parts
```

(No column name)
6

SELECT: группировка (group by)

```
select Description, count(*)
   as cnt
from parts
GROUP BY Description
```

Description	cnt
Амортизатор задний	1
Колодки задние	1

Description	cnt
Колодки передние	1
Фильтр воздушный	1
Фильтр масляный	3
Фильтр топливный	2

SELECT: функции агрегирования

```
select * from supply
```

PartID	SupplierID	Price	DeliveryTime
2	3	55	2
4	2	200	35
4	3	100	5
4	5	200	60

```
select partID, MIN(Price) as minPrice
from supply
group by partID
```

partID	minPrice
2	55
4	100

SELECT: условия на группы (having)

```
select partID, Min(Price)
from supply
group by partID
HAVING count(price) > 1
```

partId	(No column name)
4	100

```
select partID, Min(Price)
from supply
where price < 200
group by partid
having count(price) > 1
```

partId	(No column name)

3.5. Выборка данных: комбинация результатов нескольких выборок и их сортировка.

Источник: Тема 8 «SQL. Операторы DML».

Выборка данных: SELECT (SELECT statement)

- FROM – источник выборки;
- WHERE – условие выборки;
- HAVING – условие группировки;
- GROUP BY – список группировки;
- ORDER BY – сортировка результатов;
- UNION, EXCEPT, INTERCEPT – комбинирование результатов нескольких выборок.

SELECT: комбинация результатов нескольких выборок

- UNION – объединение результатов с исключением повторяющихся записей;
- UNION ALL - объединение результатов без исключения повторяющихся записей;
- EXCEPT – исключение результатов второй выборки из результатов первой;
- INTERSECT – получение пересечения результатов выборок.

Объединение результатов выборок без фильтрации повторяющихся записей: UNION ALL

```
SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]

UNION ALL

SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME013343	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный
ME215002	Фильтр масляный

Объединение результатов выборок с фильтрацией повторяющихся записей: UNION

```
SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]

UNION

SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

Исключение результатов выборок: EXCEPT

```
SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]

EXCEPT

SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]
```

PartNumber	Description

Пересечение результатов выборок: INTERSECT

```
SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]

INTERSECT

SELECT a.[PartNumber], a.[Description]
 FROM [Parts] as a, [Supply] as b
where a.[PartID] = b.[PartID]
```

PartNumber	Description
ME013343	Фильтр масляный
ME215002	Фильтр масляный

SELECT: условие выбора записей и сортировка

- сортировка (ORDER BY):

- ASC – по возрастанию (по умолчанию);
- DESC – по убыванию.

3.6. Выборка данных: соединения и их типы.

Источник: Тема 8 «SQL. Операторы DML».

SELECT: соединения (JOINS)

- внутреннее соединение:
  - INNER JOIN (JOIN)
- внешние соединения:
  - LEFT OUTER JOIN (LEFT JOIN)
  - RIGHT OUTER JOIN (RIGHT JOIN)
  - FULL OUTER JOIN (FULL JOIN)

Внутреннее соединение: INNER JOIN

```
select a.partid, a.partnumber, a.Description,
b.Price, b.DeliveryTime
from parts as a, supply as b
where a.partId = b.partid
```

```
select a.partid, a.partnumber, a.Description,
b.Price, b.DeliveryTime
from parts as a JOIN supply as b
ON a.partId = b.partid
```

partid	partnumber	Description	Price	DeliveryTime
2	ME013343	Фильтр масляный	55	2
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60

Внутреннее соединение: INNER JOIN (2)

```
select a.partid, a.Price, a.DeliveryTime,
b.partid, b.Price, b.DeliveryTime
from supply as a INNER JOIN supply as b
ON a.partId = b.partid
```

partid	Price	DeliveryTime	partid	Price	DeliveryTime
2	55	2	2	55	2
4	100	5	4	200	35
4	200	35	4	200	35
4	200	60	4	200	35
4	200	35	4	100	5
4	100	5	4	100	5
4	200	60	4	100	5
4	200	35	4	200	60
4	100	5	4	200	60
4	200	60	4	200	60

Левое соединение: LEFT OUTER JOIN

```
select a.partid, a.partnumber, a.Description,
b.Price, b.DeliveryTime
from parts as a JOIN supply as b ON a.partId = b.partid
```

partid	partnumber	Description	Price	DeliveryTime
2	ME013343	Фильтр масляный	55	2
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60

```
select a.partid, a.partnumber, a.Description,
b.Price, b.DeliveryTime
from parts as a LEFT JOIN supply as b ON a.partId = b.partid
```

partid	partnumber	Description	Price	DeliveryTime
1	MZ311785	Фильтр воздушный	NULL	NULL
2	ME013343	Фильтр масляный	55	2
3	1230A046	Фильтр масляный	NULL	NULL
4	ME215002	Фильтр масляный	200	35
4	ME215002	Фильтр масляный	100	5
4	ME215002	Фильтр масляный	200	60
6	MB220900	Фильтр топливный	NULL	NULL
7	XB220900	Фильтр топливный	NULL	NULL
8	MB891677	Амортизатор задний	NULL	NULL
9	MR389546	Колодки передние	NULL	NULL
10	MZ690170	Колодки задние	NULL	NULL

Левое соединение (2)

```
select a.*, b.SupplierID
from parts as a left join supply as b on a.partId = b.partid
```

PartID	PartNumber	Description	SupplierID
1	MZ311785	Фильтр воздушный	NULL
2	ME013343	Фильтр масляный	3
3	1230A046	Фильтр масляный	NULL
4	ME215002	Фильтр масляный	2
4	ME215002	Фильтр масляный	3
4	ME215002	Фильтр масляный	5
6	MB220900	Фильтр топливный	NULL
7	XB220900	Фильтр топливный	NULL

PartID	PartNumber	Description	SupplierID
8	MB891677	Амортизатор задний	NULL
9	MR389546	Колодки передние	NULL
10	MZ690170	Колодки задние	NULL

```
select a.partnumber, count(b.price) as cnt            -- count(distinct
b.price) as cnt
from parts as a left join supply as b on a.partId = b.partid
group by a.partnumber
```

Partnumber	cnt
1230A046	0
MB220900	0
MB891677	0
ME013343	1
ME215002	3
MR389546	0
MZ311785	0
MZ690170	0
XB220900	0

Правое соединение: RIGHT OUTER JOIN

```
select a.partid, a.partnumber, b.Price, b.DeliveryTime, b.SupplierID
from parts as a JOIN supply as b ON a.partId = b.partid
```

partid	partnumber	Price	DeliveryTime	SupplierID
2	ME013343	55	2	3
4	ME215002	200	35	2

partid	partnumber	Price	DeliveryTime	SupplierID
4	ME215002	100	5	3
4	ME215002	200	60	5

```
select b.PartID, b.Price, b.DeliveryTime, c.SupplierCode
from supply as b RIGHT JOIN suppliers as c ON b.supplierId = c.supplierid
```

PartID	Price	DeliveryTime	SupplierCode
NULL	NULL	NULL	EXIST
4	200	35	AVTORIF
2	55	2	UAE
4	100	5	UAE
4	200	60	GER
NULL	NULL	NULL	JAP

3.7. Вставка данных: оператор INSERT

Источник: Тема 8 «SQL. Операторы DML».

Вставка строк: INSERT (INSERT statement)

- вставка единственной строки:
  - с указанием всех значений;
  - с указанием некоторых значений;
  - с указанием значений в порядке, отличающемся от порядка полей таблицы;
- вставка нескольких строк:
  - с явным указанием значений;
  - с использованием результатов выборки из другой таблицы.

INSERT: вставка с указанием всех значений

```
INSERT INTO Production.UnitMeasure
VALUES (N'FT', N'Feet', '20080414');
GO

INSERT INTO Production.UnitMeasure
VALUES (N'FT2', N'Square Feet ', '20080923'),
       (N'Y', N'Yards', '20080923'),
       (N'Y3', N'Cubic Yards', '20080923');
GO

INSERT INTO Production.UnitMeasure (Name, UnitMeasureCode,
   ModifiedDate)
VALUES (N'Square Yards', N'Y2', GETDATE())
GO
```

INSERT: вставка в таблицу со значениями по умолчанию

```
IF OBJECT_ID ('dbo.T1', 'U') IS NOT NULL
DROP TABLE dbo.T1;
GO

CREATE TABLE dbo.T1 (
   column_1 AS 'Computed column ' + column_2,
   column_2 varchar(30) CONSTRAINT default_name DEFAULT ('my column
default'),
   column_3 rowversion,
   column_4 varchar(40) NULL
   );
GO

INSERT INTO dbo.T1 (column_4) VALUES ('Explicit value');
INSERT INTO dbo.T1 (column_2, column_4) VALUES ('Explicit value', 'Explicit
value');
INSERT INTO dbo.T1 (column_2) VALUES ('Explicit value');
INSERT INTO T1 DEFAULT VALUES;
GO
```

INSERT: вставка в таблицу с автоинкрементным полем (IDENTITY)

```
IF OBJECT_ID ('dbo.T1', 'U') IS NOT NULL
DROP TABLE dbo.T1;
GO

CREATE TABLE dbo.T1 (
   column_1 int IDENTITY,
   column_2 VARCHAR(30)
   );
GO

INSERT T1 VALUES ('Row #1');
INSERT T1 (column_2) VALUES ('Row #2');
GO

SET IDENTITY_INSERT T1 ON;
GO
INSERT INTO T1 (column_1,column_2) VALUES (-99, 'Explicit identity value');
GO
```

INSERT: вставка результатов выборки (SELECT)

```
IF OBJECT_ID ('dbo.EmployeeSales', 'U') IS NOT NULL
DROP TABLE dbo.EmployeeSales;
GO

CREATE TABLE dbo.EmployeeSales (
   DataSource varchar(20) NOT NULL,
   BusinessEntityID varchar(11) NOT NULL,
   LastName varchar(40) NOT NULL,
   SalesDollars money NOT NULL
   );
GO

INSERT INTO dbo.EmployeeSales
SELECT 'SELECT', sp.BusinessEntityID, c.LastName, sp.SalesYTD
FROM Sales.SalesPerson AS sp
   INNER JOIN Person.Person AS c ON sp.BusinessEntityID = c.BusinessEntityID
WHERE sp.BusinessEntityID LIKE '2%'
ORDER BY sp.BusinessEntityID, c.LastName;
GO
```

3.8. Обновление данных: оператор UPDATE. Условие отбора записей. Значение NULL и трехзначная логика.

Источник: Тема 8 «SQL. Операторы DML».

Изменение строк: UPDATE (UPDATE statement)

```
UPDATE Person.Address
SET ModifiedDate = GETDATE();            -- SET ModifiedDate = DEFAULT;
GO

UPDATE Sales.SalesPerson
SET Bonus = 6000, CommissionPct = .10, SalesQuota = NULL;
GO

UPDATE Production.Product
SET Color = N'Metallic Red'
WHERE Name LIKE N'Road-250%' AND Color = N'Red';
GO

UPDATE Production.Product
SET ListPrice = ListPrice * 2;
GO

DECLARE @NewPrice int = 10;
UPDATE Production.Product
SET ListPrice += @NewPrice
GO
```

Изменение строк: UPDATE (2)

```
UPDATE TOP (10) HumanResources.Employee
SET VacationHours = VacationHours * 1.25 ;
GO

UPDATE HumanResources.Employee
SET VacationHours = VacationHours + 8
   FROM (SELECT TOP 10 BusinessEntityID
                    FROM HumanResources.Employee
                    ORDER BY HireDate ASC) AS th
   WHERE HumanResources.Employee.BusinessEntityID = th.BusinessEntityID;
GO

UPDATE Sales.SalesPerson
```

```
SET SalesYTD = SalesYTD +
  (SELECT SUM(so.SubTotal)
   FROM Sales.SalesOrderHeader AS so
   WHERE so.OrderDate = (SELECT MAX(OrderDate)
               FROM Sales.SalesOrderHeader AS so2
               WHERE so2.SalesPersonID = so.SalesPersonID)
   AND Sales.SalesPerson.BusinessEntityID = so.SalesPersonID
   GROUP BY so.SalesPersonID);
GO
```

SELECT: условие выбора записей и сортировка

- условие выбора записей (WHERE…) – задается с помощью логического выражения, формируемого с помощью:
  - операторов сравнения (>, <, =, <>, <=, >=);
  - логических операторов (AND, OR, NOT);
  - задания диапазона (*value* BETWEEN *min* AND *max*);
  - условия принадлежности множеству значений (*value* IN (*list*), список может формироваться с помощью вложенных запросов);
  - условия непустого множества строк, возвращаемого в результате выполнения подзапроса (EXISTS (subquery));
  - задания шаблона для строковых значений (value LIKE '…')
    - символ % является шаблоном подстроки;
    - символ _ является шаблоном единственного символа;
    - [],[^] позволяют указать/ограничить допустмый набор/диапазон символов;
  - определения наличия неопределенного значения (IS NULL | IS NOT NULL)
- сортировка (ORDER BY):
  - ASC – по возрастанию (по умолчанию);
  - DESC – по убыванию.

SELECT: неопределенные значения (NULL)

```
SELECT *
FROM suppliers
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
2	AVTORIF	NULL
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

```
set ansi_nulls off
SELECT * from suppliers
where Name = null
```

SupplierID	SupplierCode	Name
2	AVTORIF	NULL

```
SELECT * from suppliers
where Name <> null
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

```
set ansi_nulls on
SELECT * from suppliers
where Name = null
```

SupplierID	SupplierCode	Name

```
SELECT * from suppliers
where Name <> null
```

SupplierID	SupplierCode	Name

Значения NULL и трехзначная логика (3VL, three-value logic)

Image — Значения NULL и трехзначная логика (3VL, three-value logic)

SELECT: **неопределенные значения** (NULL)

```
SELECT *
FROM suppliers
WHERE Name IS NULL
```

SupplierID	SupplierCode	Name
2	AVTORIF	NULL

```
SELECT *
FROM suppliers
WHERE Name IS NOT NULL
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

3.9. Удаление данных: оператор DELETE. Условие отбора записей. Значение NULL и трехзначная логика.

Источник: Тема 8 «SQL. Операторы DML».

Удаление строк: DELETE (DELETE statement)

```
DELETE FROM Sales.SalesPersonQuotaHistory;
GO
-- TRUNCATE TABLE Sales.SalesPersonQuotaHistory;
-- GO

DELETE FROM Production.ProductCostHistory
WHERE StandardCost > 1000.00;
GO

DELETE FROM Sales.SalesPersonQuotaHistory
WHERE BusinessEntityID IN
   (SELECT BusinessEntityID FROM Sales.SalesPerson WHERE SalesYTD >
2500000.00);
GO

DELETE FROM Sales.SalesPersonQuotaHistory
FROM Sales.SalesPersonQuotaHistory AS spqh
   INNER JOIN Sales.SalesPerson AS sp ON spqh.BusinessEntityID =
sp.BusinessEntityID
   WHERE sp.SalesYTD > 2500000.00;
GO
```

SELECT: **условие выбора записей и сортировка**

- условие выбора записей (WHERE…) – задается с помощью логического выражения, формируемого с помощью:
  - операторов сравнения (>, <, =, <>, <=, >=);
  - логических операторов (AND, OR, NOT);
  - задания диапазона (*value* BETWEEN *min* AND *max*);
  - условия принадлежности множеству значений (*value* IN (*list*), список может формироваться с помощью вложенных запросов);
  - условия непустого множества строк, возвращаемого в результате выполнения подзапроса (EXISTS (subquery));
  - задания шаблона для строковых значений (value LIKE ‘…’)
    - символ % является шаблоном подстроки;
    - символ _ является шаблоном единственного символа;
    - [],[^] позволяют указать/ограничить допустмый набор/диапазон символов;

    - определения наличия неопределенного значения (IS NULL | IS NOT NULL)
- сортировка (ORDER BY):
    - ASC – по возрастанию (по умолчанию);
    - DESC – по убыванию.

SELECT: неопределенные значения (NULL)

```
SELECT *
FROM suppliers
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
2	AVTORIF	NULL
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония

```
set ansi_nulls off
SELECT * from suppliers
where Name = null
```

SupplierID	SupplierCode	Name
2	AVTORIF	NULL

```
SELECT * from suppliers
where Name <> null
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
3	UAE	Эмираты
5	GER	Германия

SupplierID	SupplierCode	Name
6	JAP	Япония

```
set ansi_nulls on
SELECT * from suppliers
where Name = null
```

SupplierID	SupplierCode	Name

```
SELECT * from suppliers
where Name <> null
```

SupplierID	SupplierCode	Name

Значения NULL и трехзначная логика (3VL, three-value logic)

Image — Значения NULL и трехзначная логика (3VL, three-value logic)

SELECT: неопределенные значения (NULL)

```
SELECT *
FROM suppliers
WHERE Name IS NULL
```

SupplierID	SupplierCode	Name
2	AVTORIF	NULL

```
SELECT *
FROM suppliers
WHERE Name IS NOT NULL
```

SupplierID	SupplierCode	Name
1	EXIST	ООО Эксист
3	UAE	Эмираты
5	GER	Германия
6	JAP	Япония